

INTELIGÊNCIA ARTIFICIAL

PARTE 4

Buscas em Árvores Mínimas

- Kruskal
- Dijkstra



1

Algoritmo de Kruskal

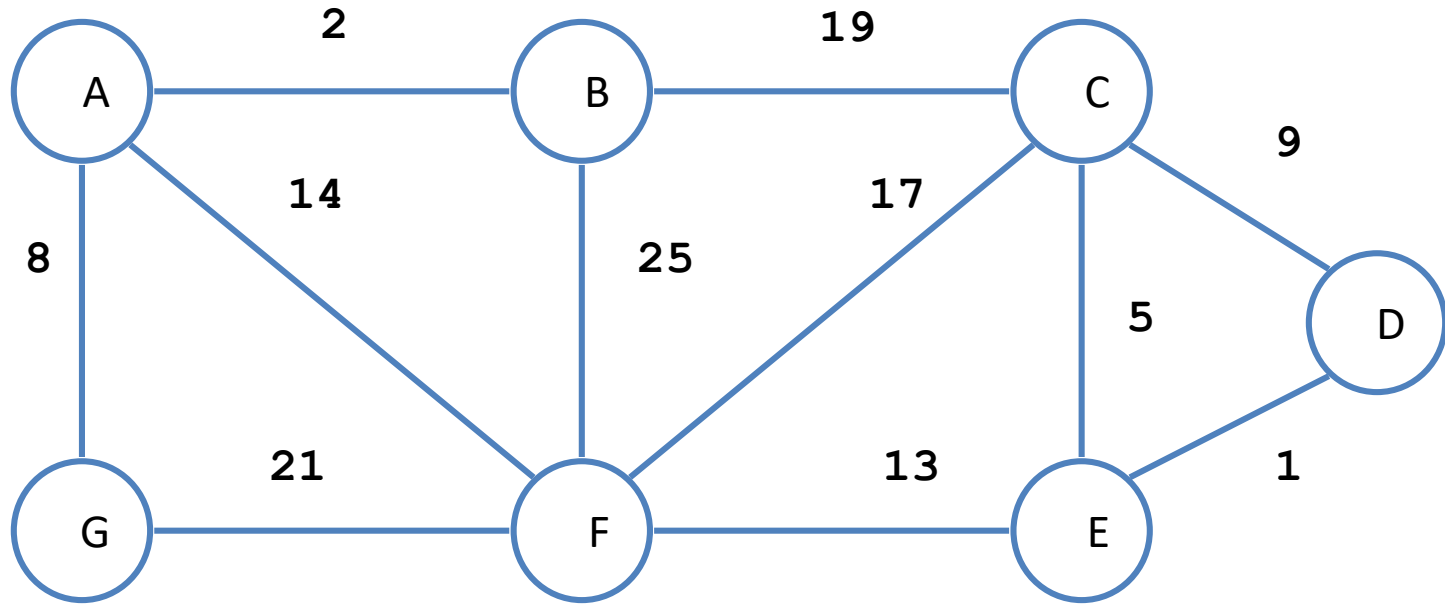
1 – Algoritmo de Kruskal

Características:

- É um algoritmo de busca, guloso, que se baseia no algoritmo de árvore espalhada de peso mínimo.
- Encontra uma aresta segura para adicionar à floresta crescente encontrando, de todas as arestas que conectam duas árvores quaisquer, uma aresta de peso mínimo.
- Encontra um subconjunto das arestas que forma uma árvore com todos os vértices, onde o peso total, dado pela soma dos pesos das arestas da árvore, é minimizado.

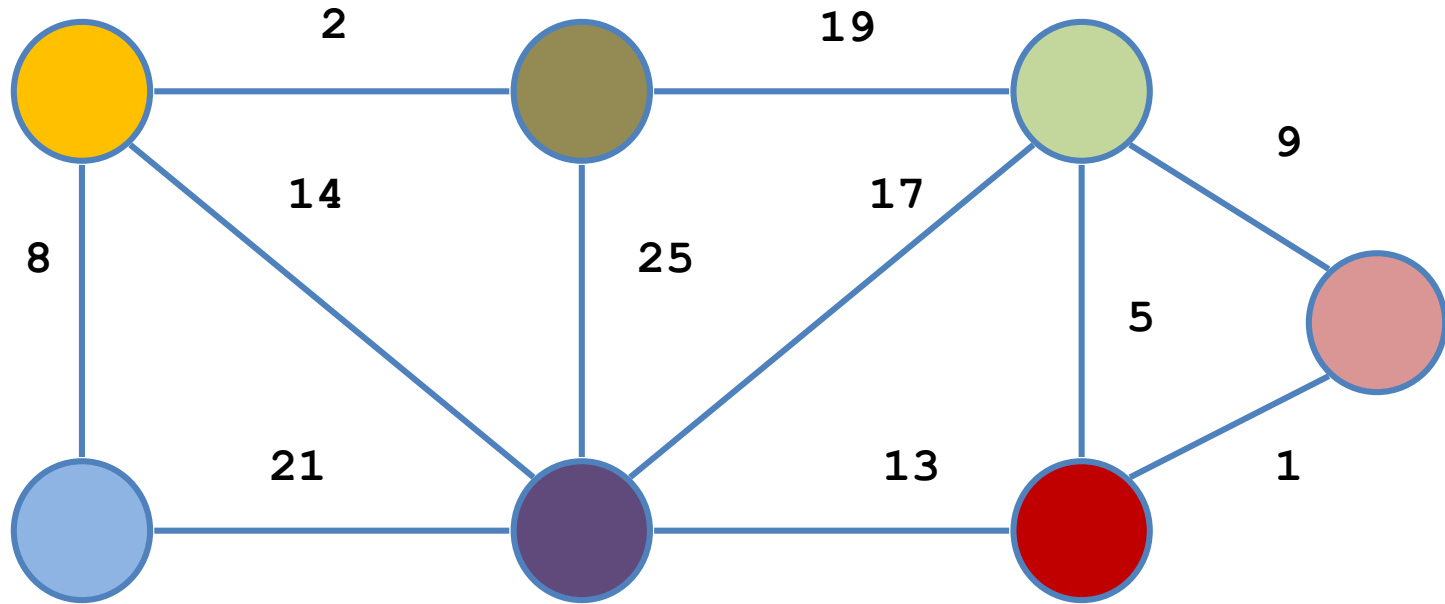
1 – Algoritmo de Kruskal

Exemplo:



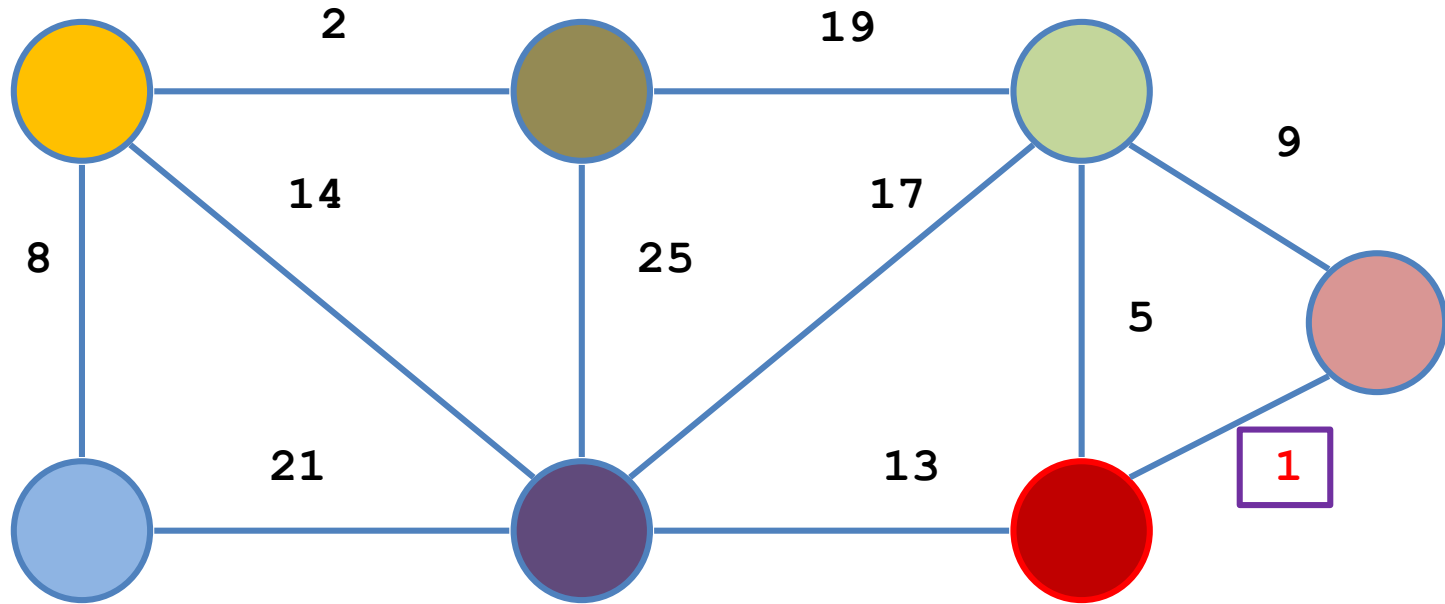
1 – Algoritmo de Kruskal

Exemplo:



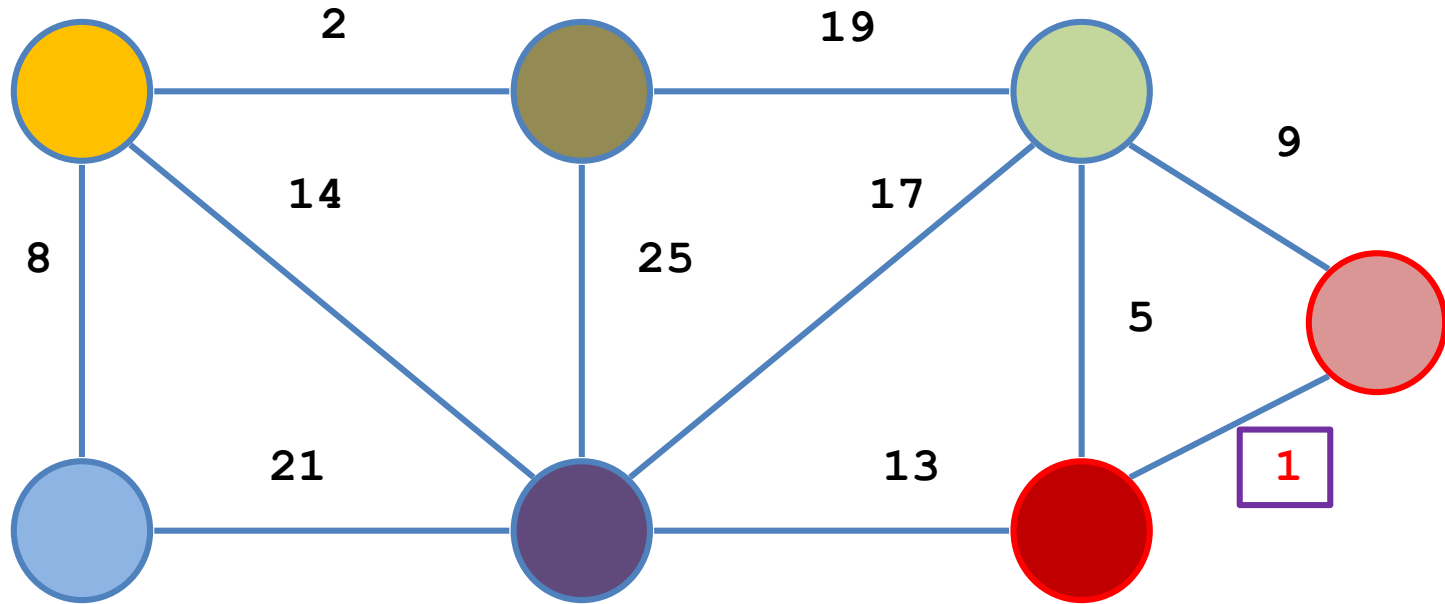
1 – Algoritmo de Kruskal

Exemplo:



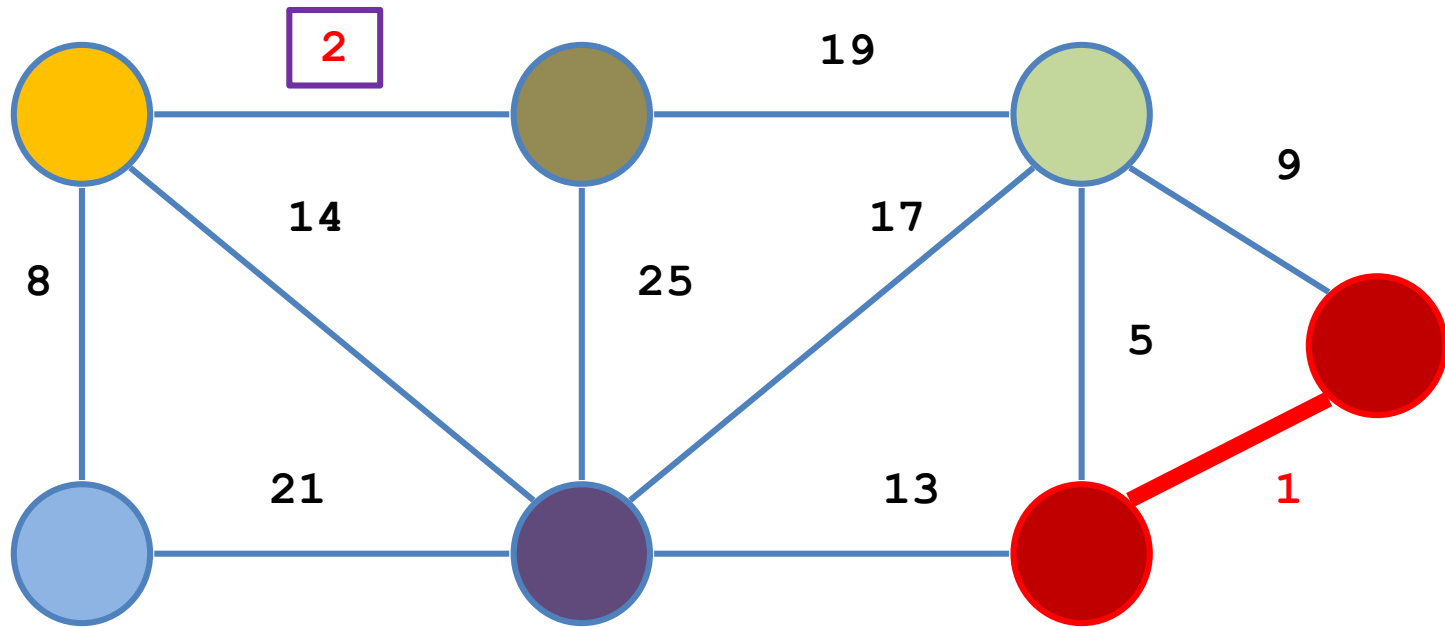
1 – Algoritmo de Kruskal

Exemplo:



1 – Algoritmo de Kruskal

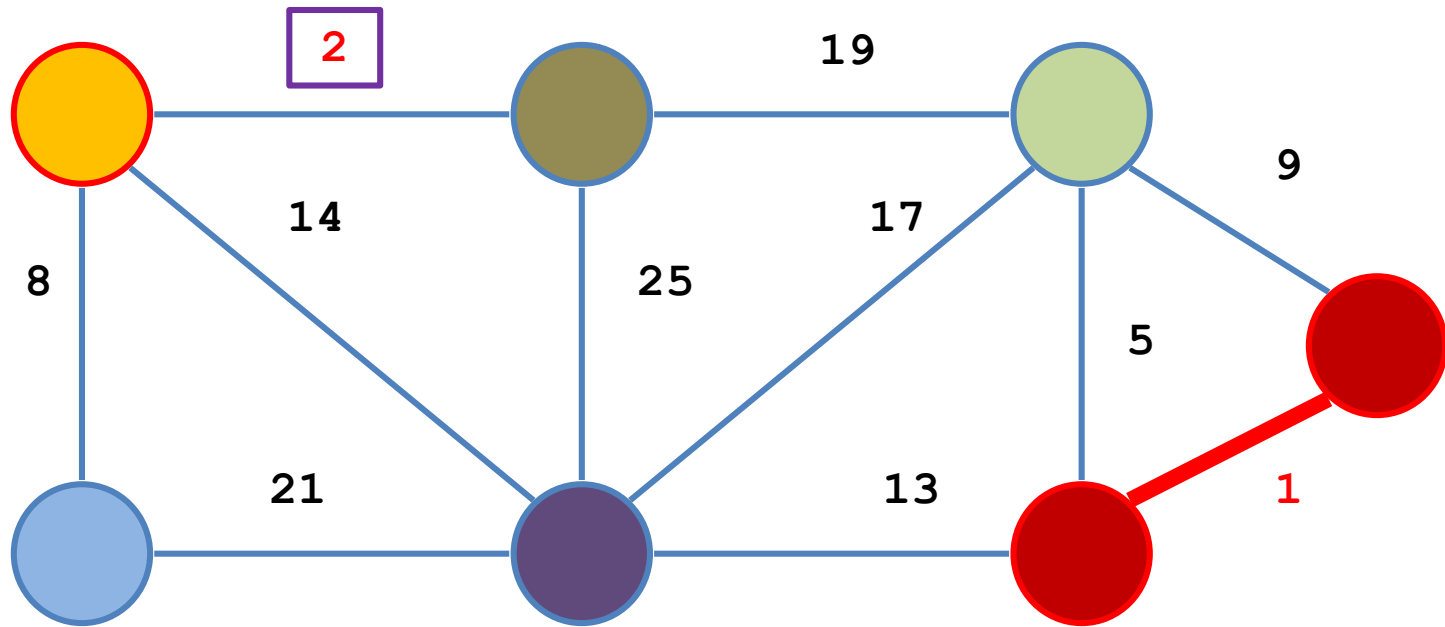
Exemplo:



$$w_{T_1} = 1$$

1 – Algoritmo de Kruskal

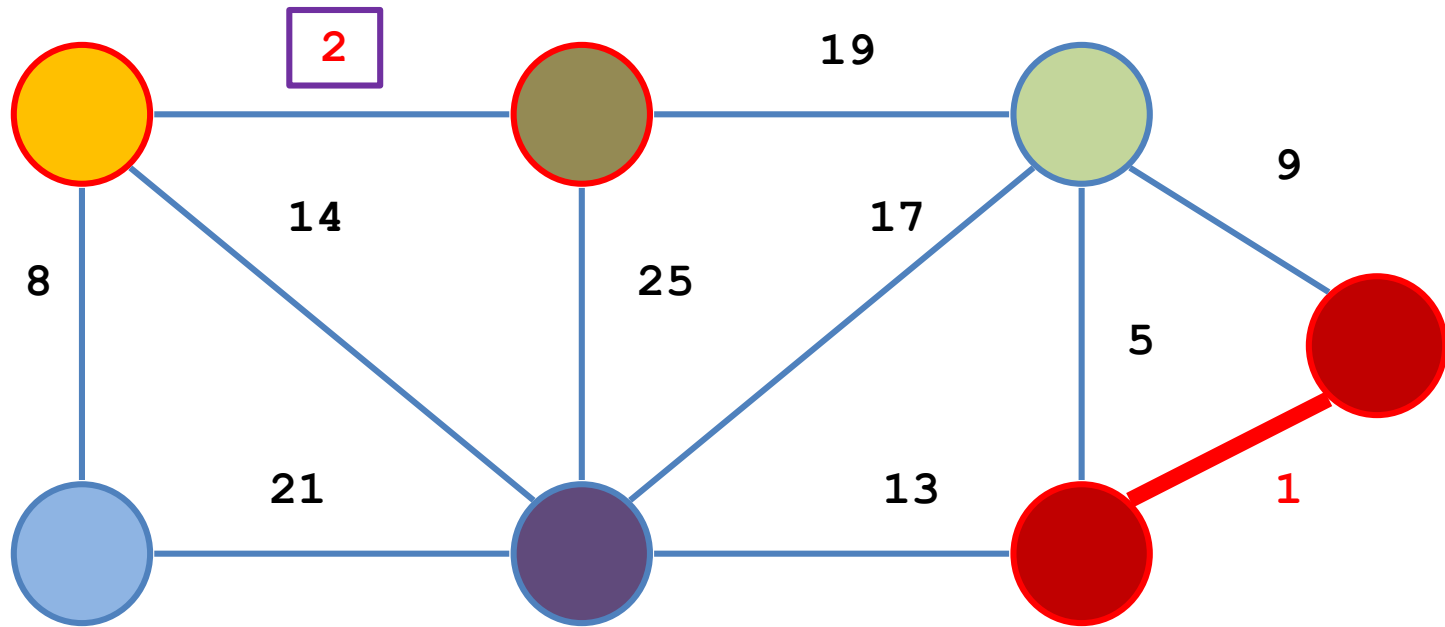
Exemplo:



$$w_{T_1} = 1$$

1 – Algoritmo de Kruskal

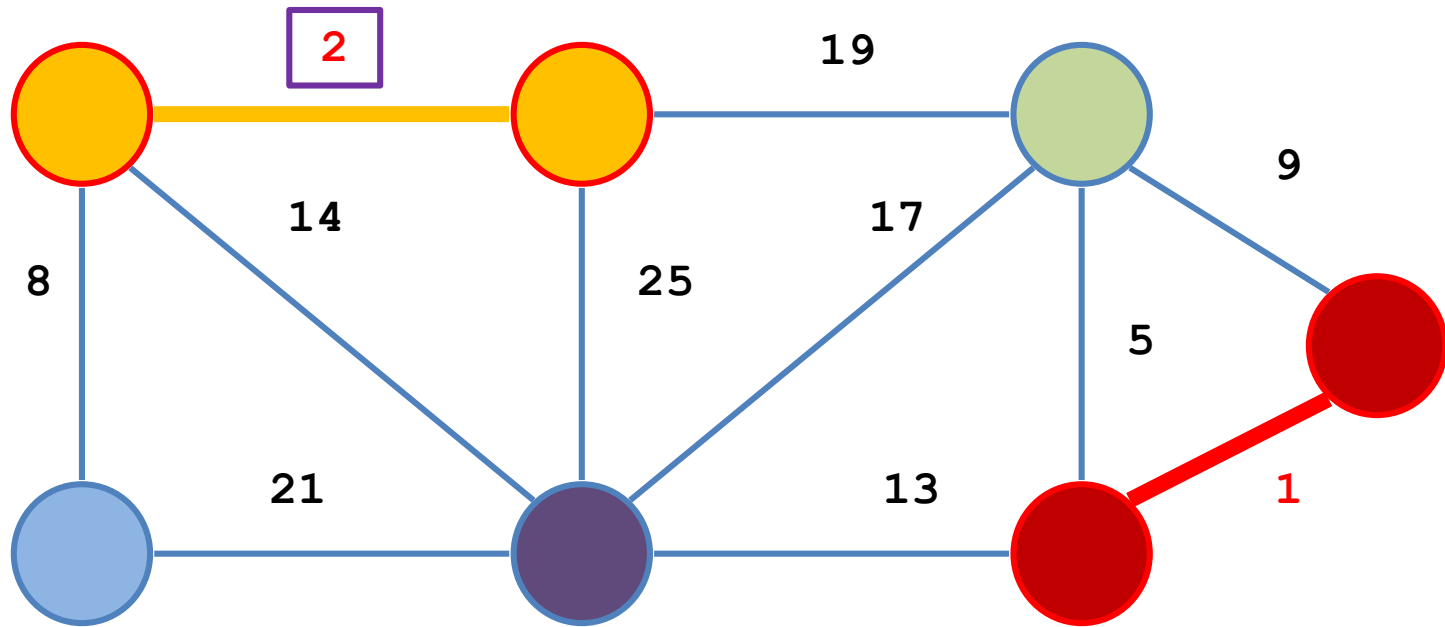
Exemplo:



$$w_{T_1} = 1$$

1 – Algoritmo de Kruskal

Exemplo:

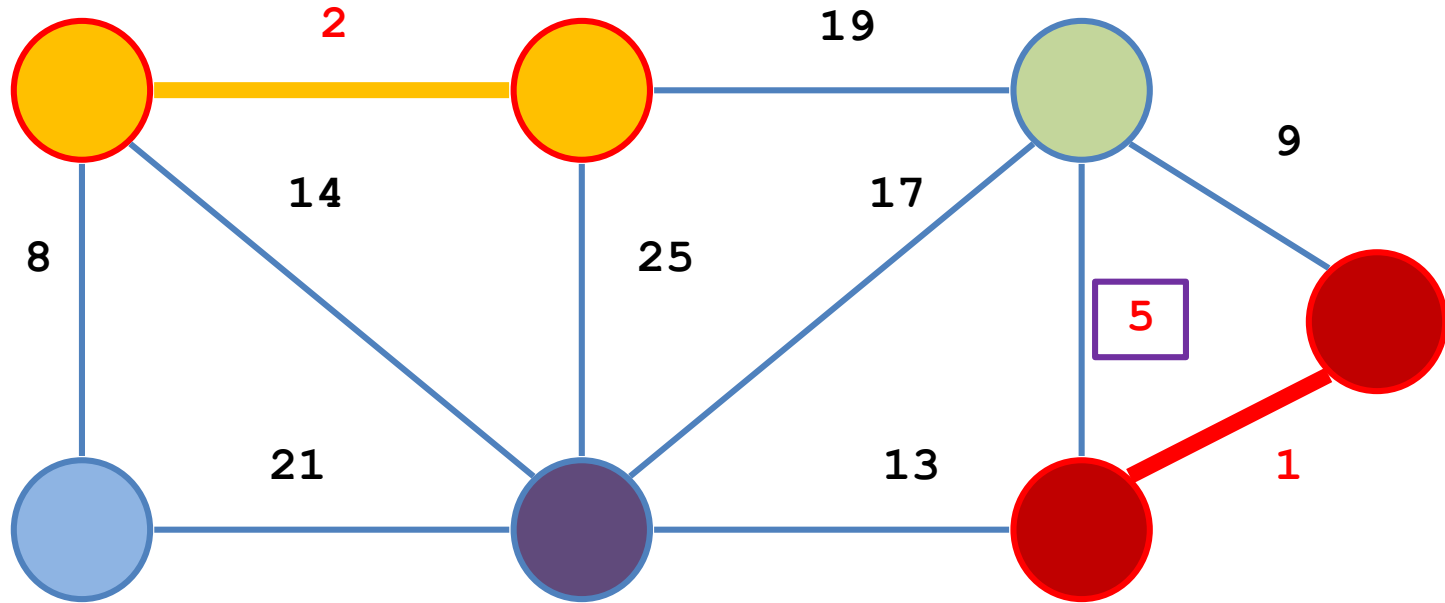


$$w_{T1} = 1$$

$$w_{T2} = 2$$

1 – Algoritmo de Kruskal

Exemplo:

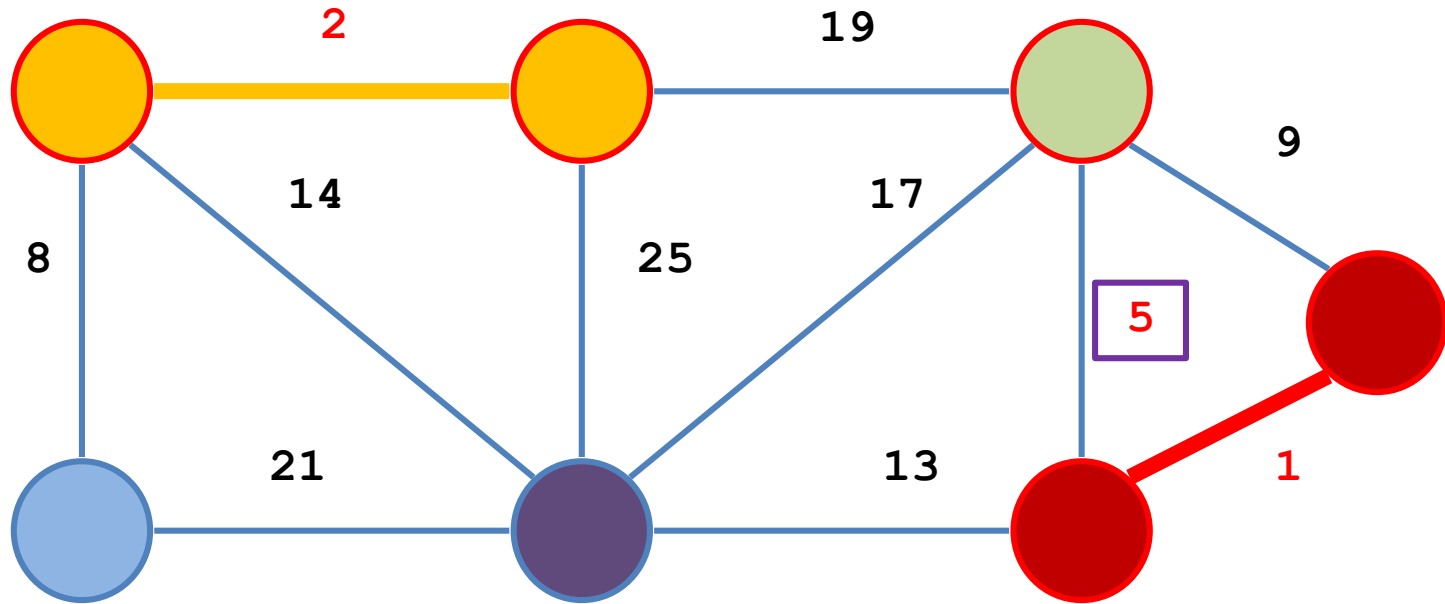


$$w_{T1} = 1$$

$$w_{T2} = 2$$

1 – Algoritmo de Kruskal

Exemplo:

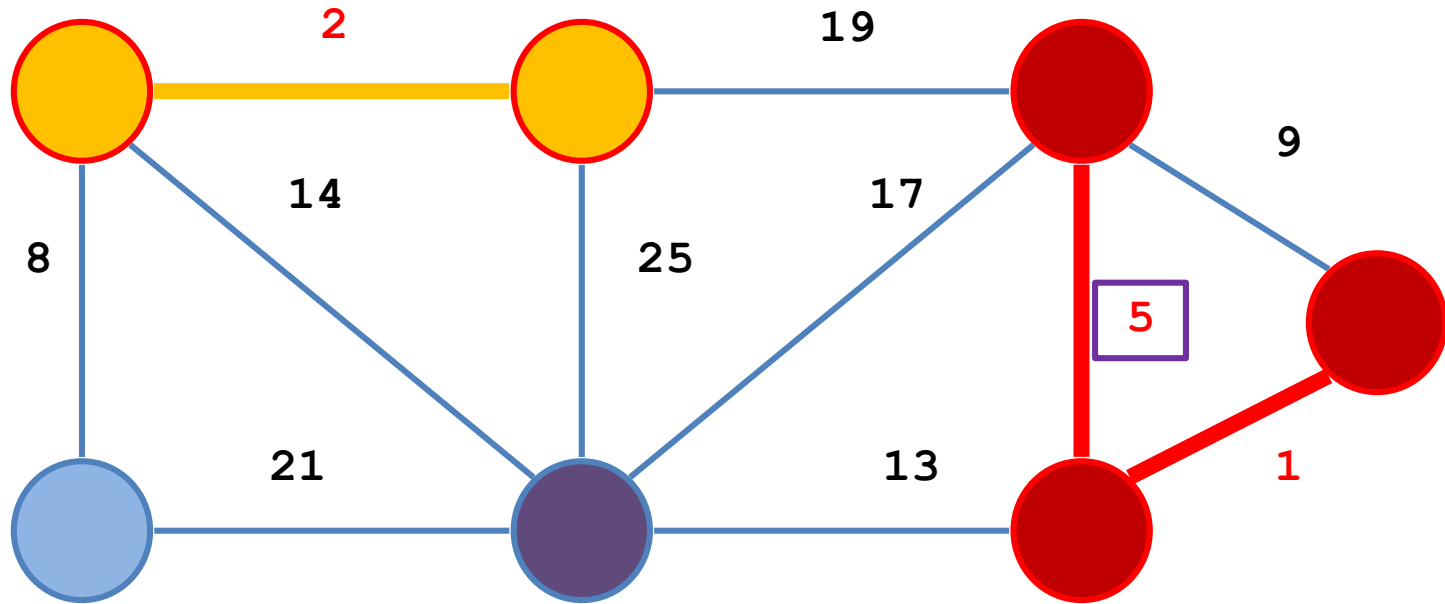


$$w_{T_1} = 1$$

$$w_{T_2} = 2$$

1 – Algoritmo de Kruskal

Exemplo:

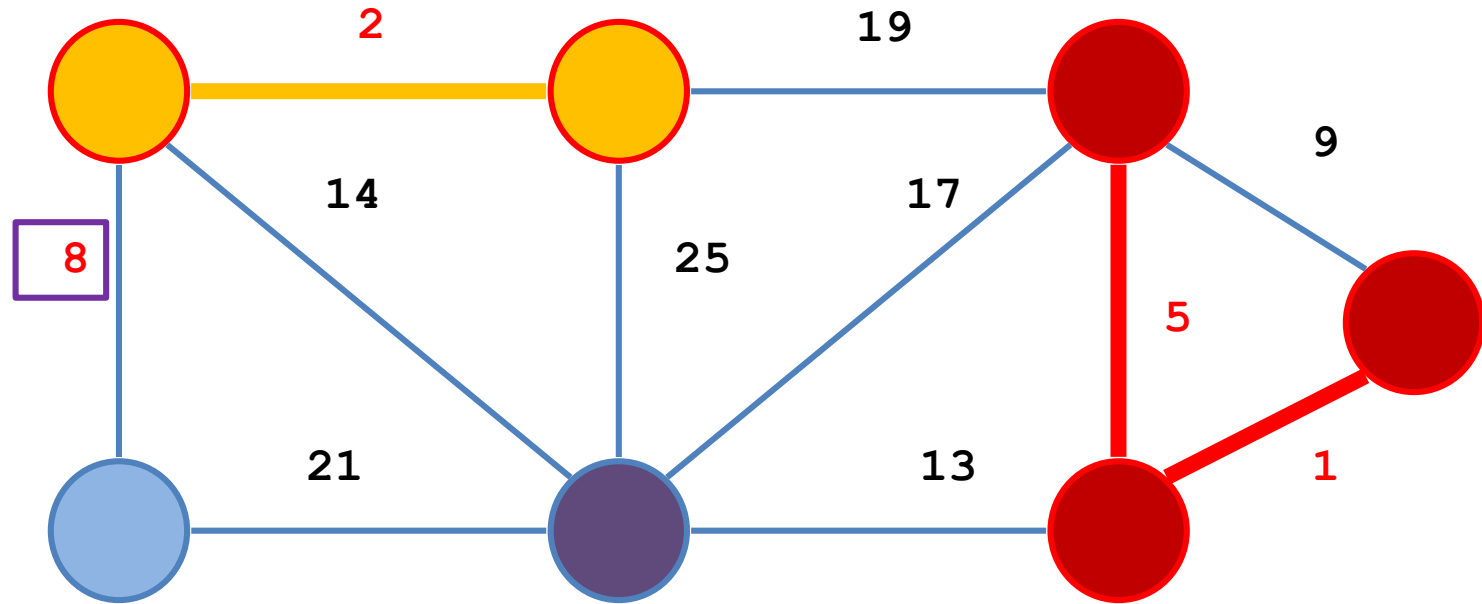


$$w_{T1} = 1 + 5$$

$$w_{T2} = 2$$

1 – Algoritmo de Kruskal

Exemplo:

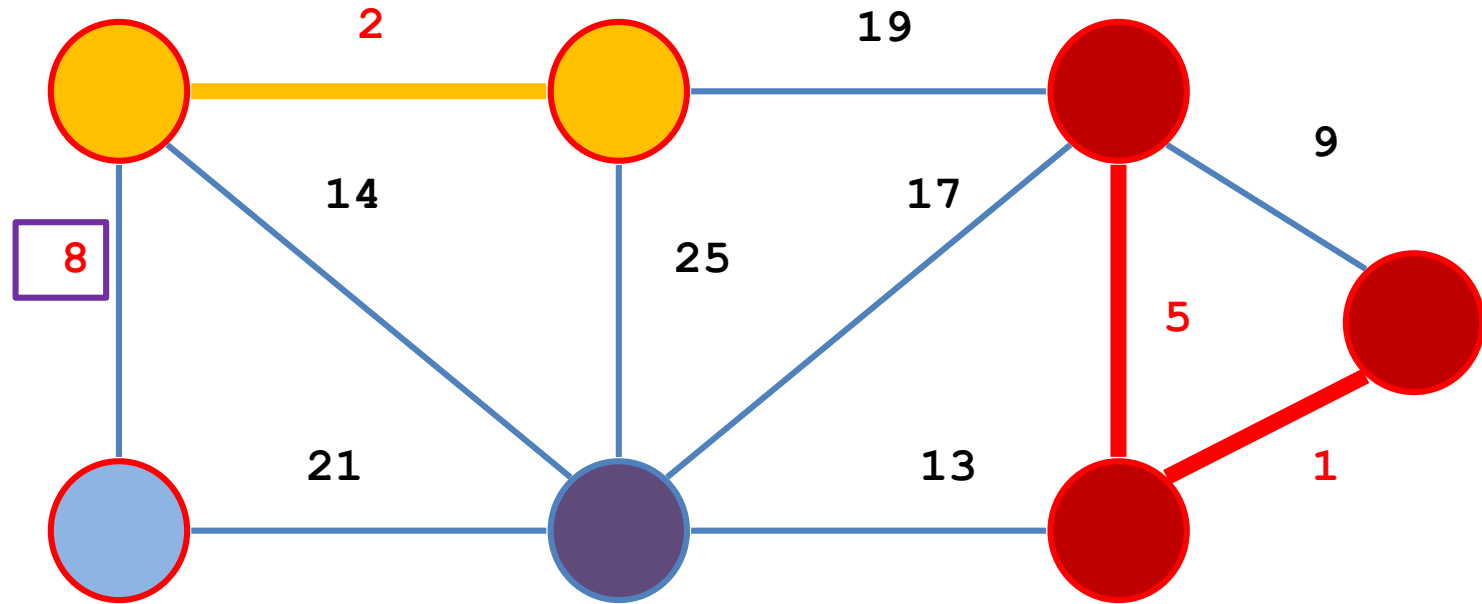


$$w_{T1} = 1 + 5$$

$$w_{T2} = 2$$

1 – Algoritmo de Kruskal

Exemplo:

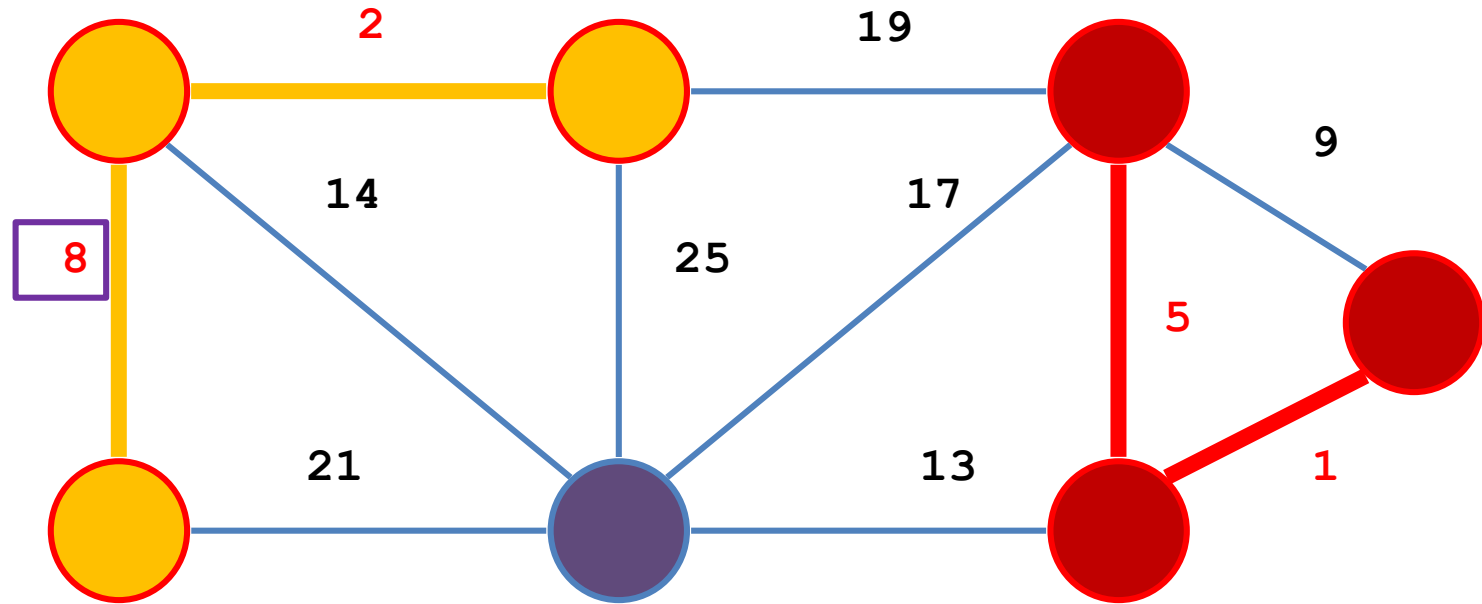


$$w_{T1} = 1 + 5$$

$$w_{T2} = 2$$

1 – Algoritmo de Kruskal

Exemplo:

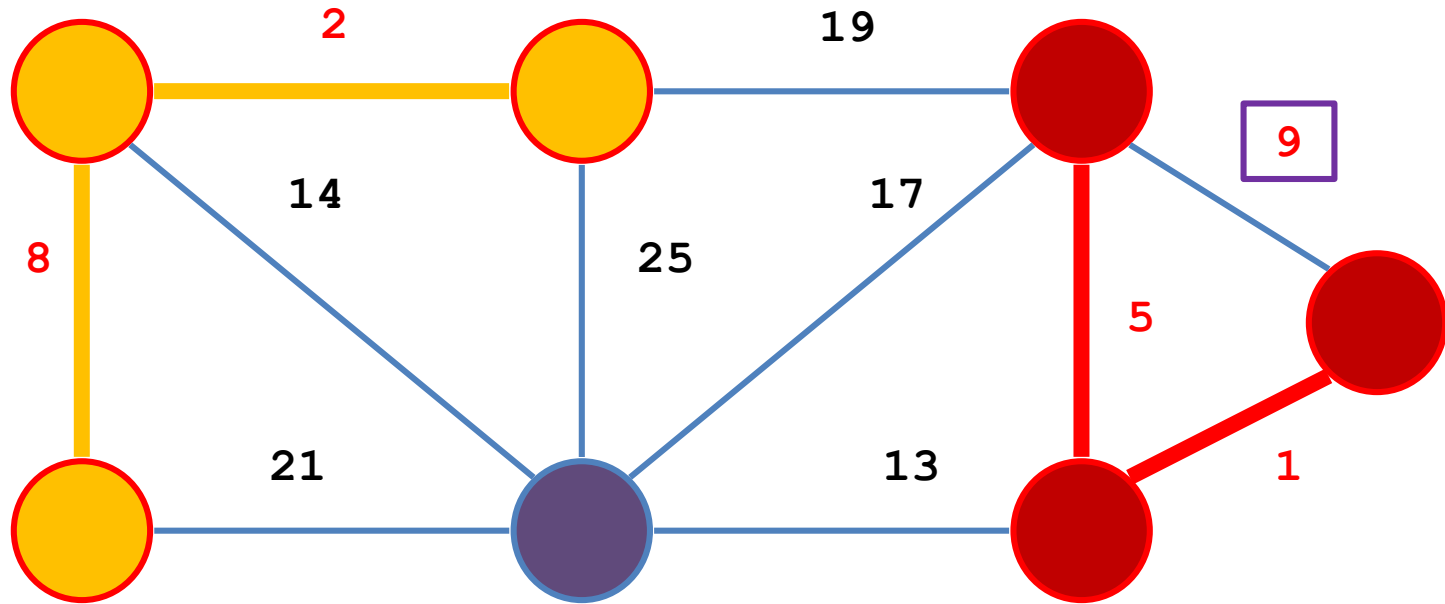


$$w_{T_1} = 1 + 5$$

$$w_{T_2} = 2 + 8$$

1 – Algoritmo de Kruskal

Exemplo:

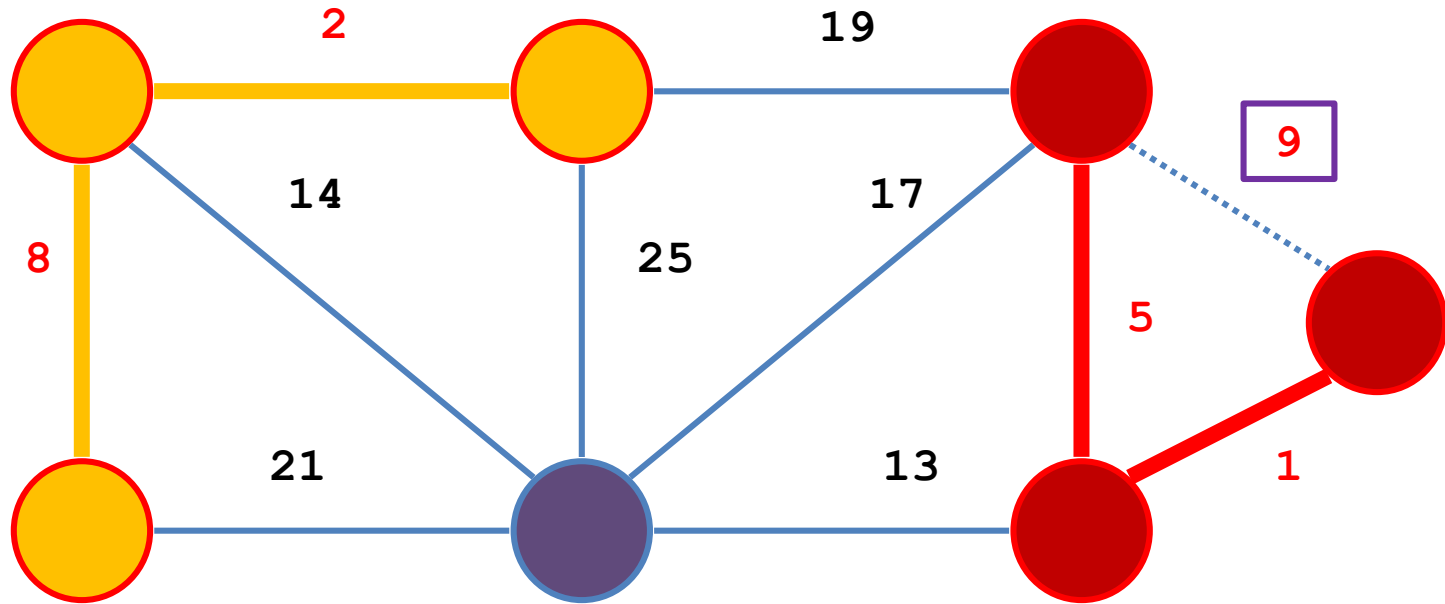


$$w_{T_1} = 1 + 5$$

$$w_{T_2} = 2 + 8$$

1 – Algoritmo de Kruskal

Exemplo:

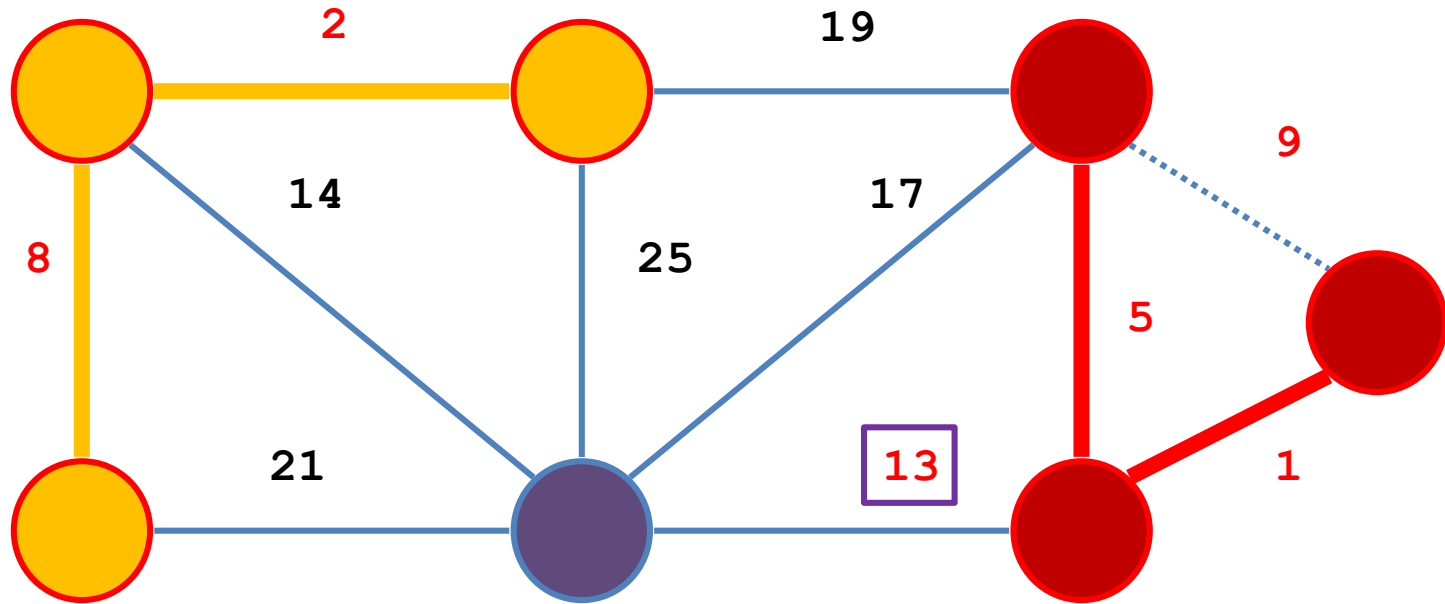


$$w_{T_1} = 1 + 5$$

$$w_{T_2} = 2 + 8$$

1 – Algoritmo de Kruskal

Exemplo:

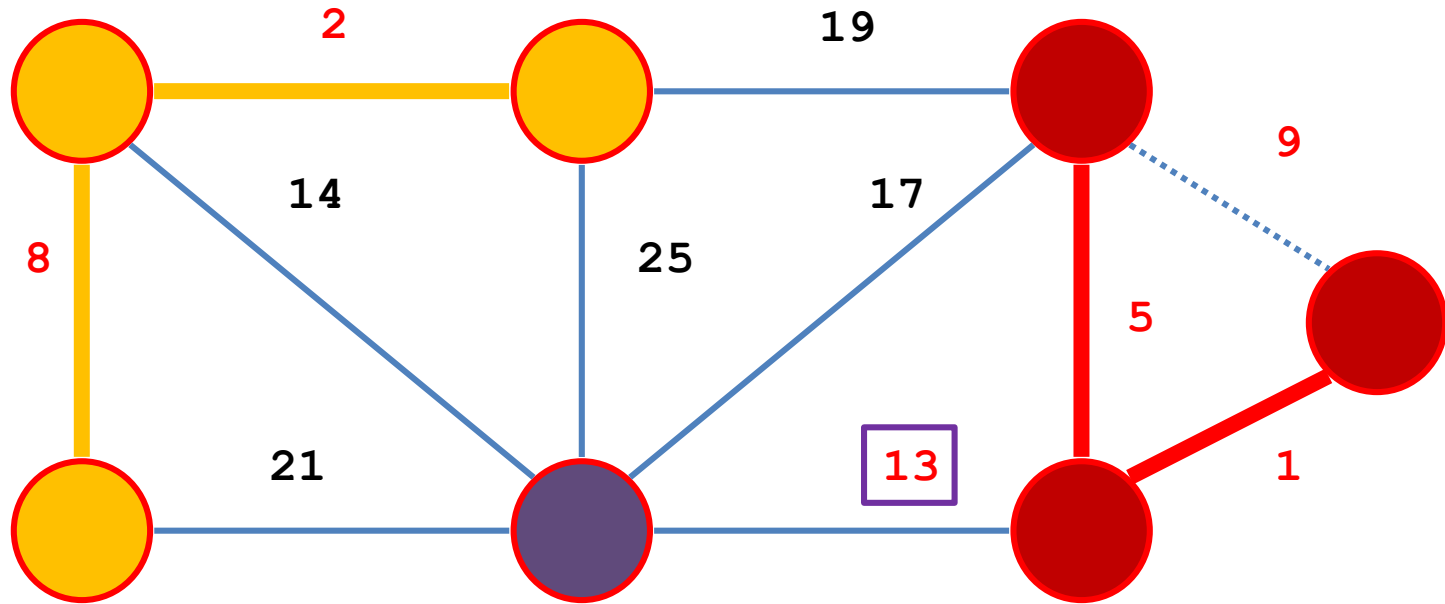


$$w_{T_1} = 1 + 5$$

$$w_{T_2} = 2 + 8$$

1 – Algoritmo de Kruskal

Exemplo:

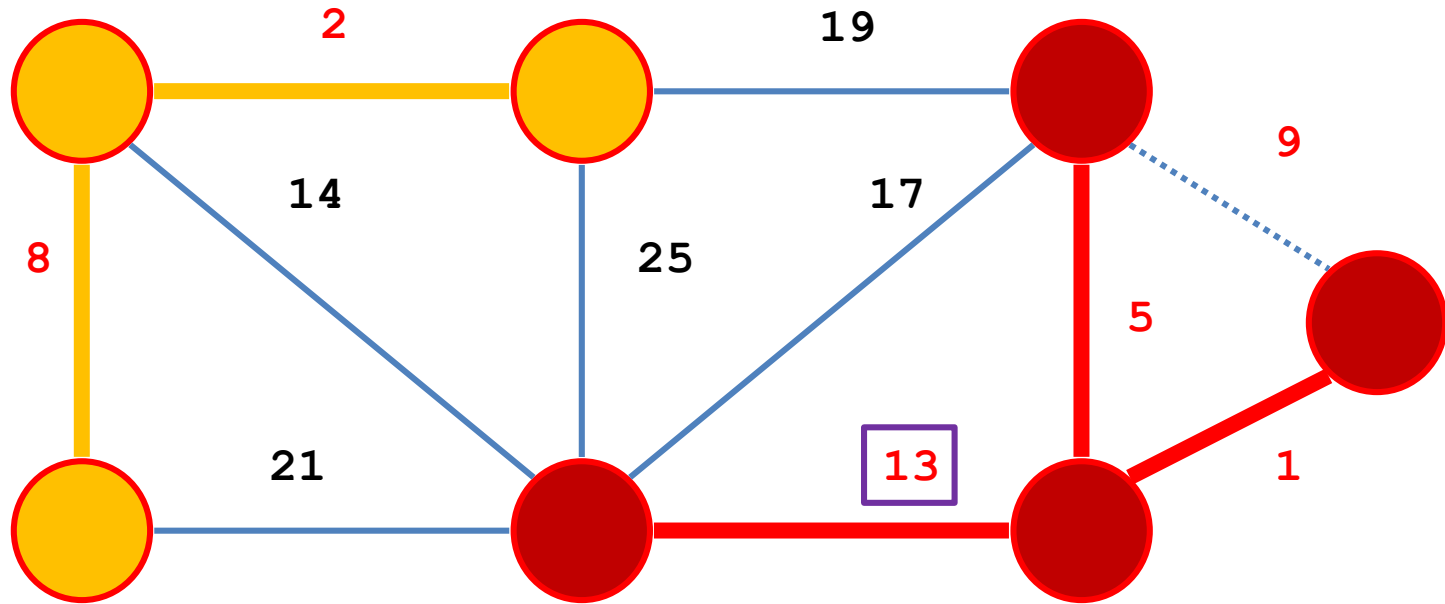


$$w_{T1} = 1 + 5$$

$$w_{T2} = 2 + 8$$

1 – Algoritmo de Kruskal

Exemplo:

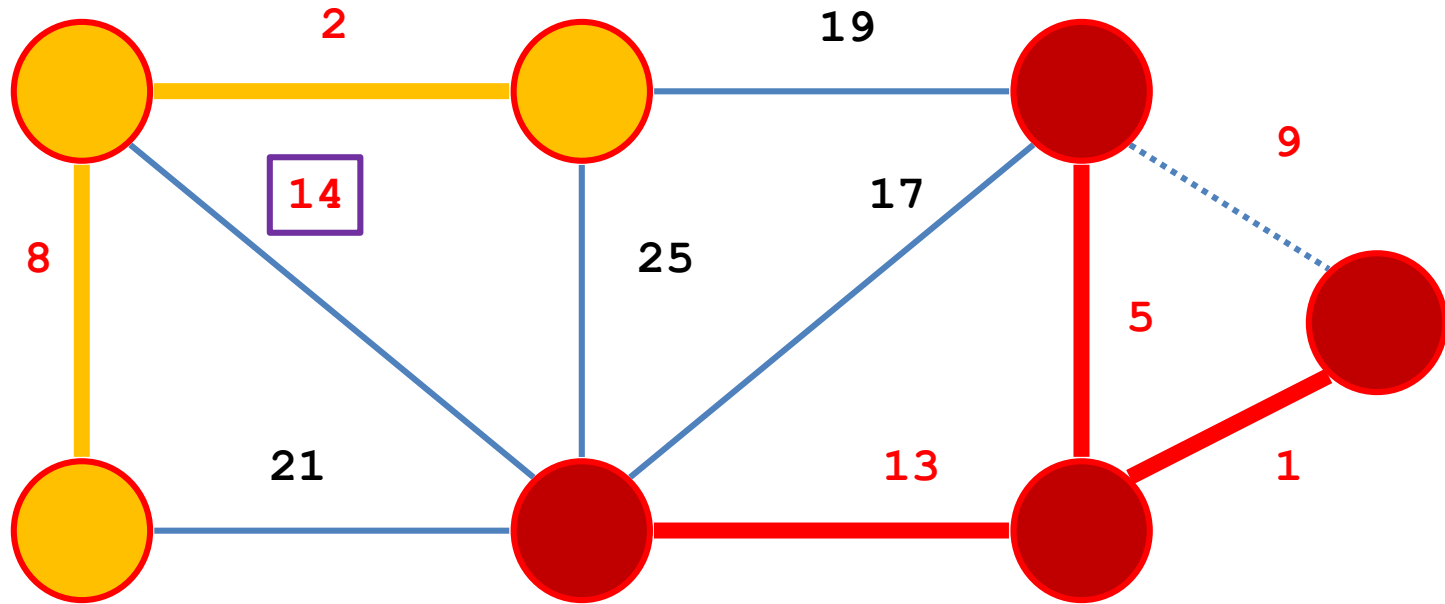


$$w_{T_1} = 1 + 5 + 13$$

$$w_{T_2} = 2 + 8$$

1 – Algoritmo de Kruskal

Exemplo:

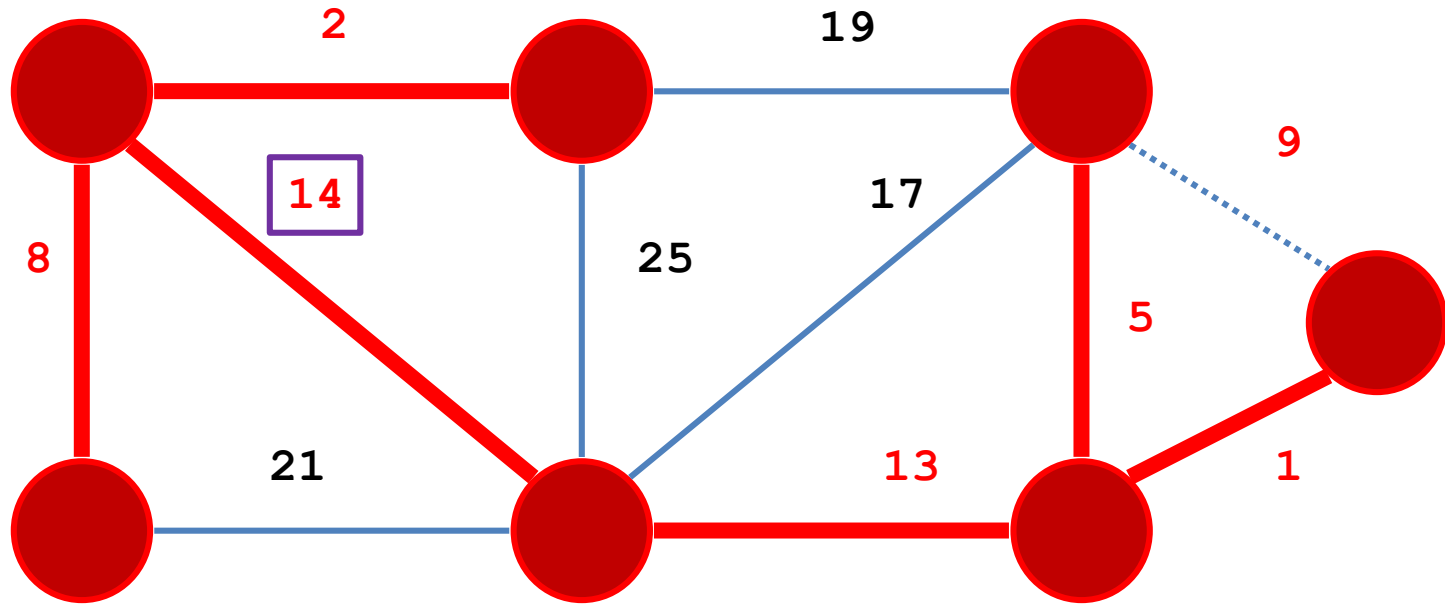


$$w_{T1} = 1 + 5 + 13$$

$$w_{T2} = 2 + 8$$

1 – Algoritmo de Kruskal

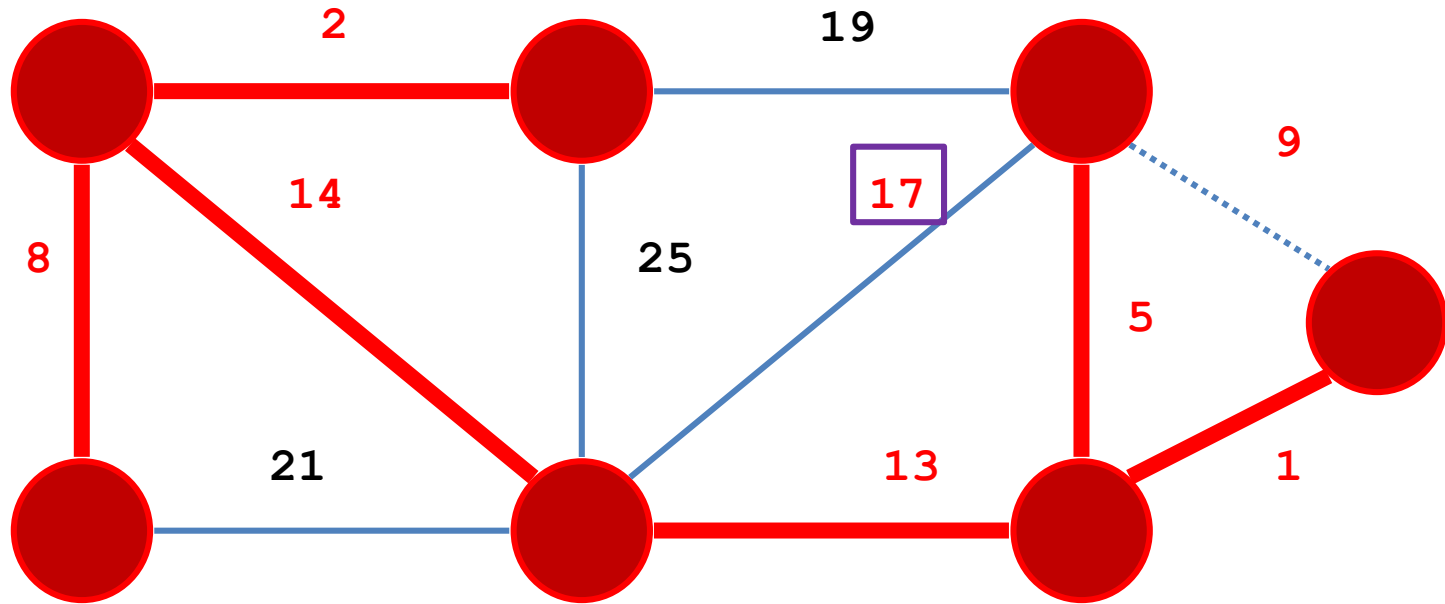
Exemplo:



$$\left. \begin{array}{l} w_{T_1} = 1 + 5 + 13 \\ w_{T_2} = 2 + 8 \end{array} \right\} w_{T_1 + T_2} = 19 + 10 + 14$$

1 – Algoritmo de Kruskal

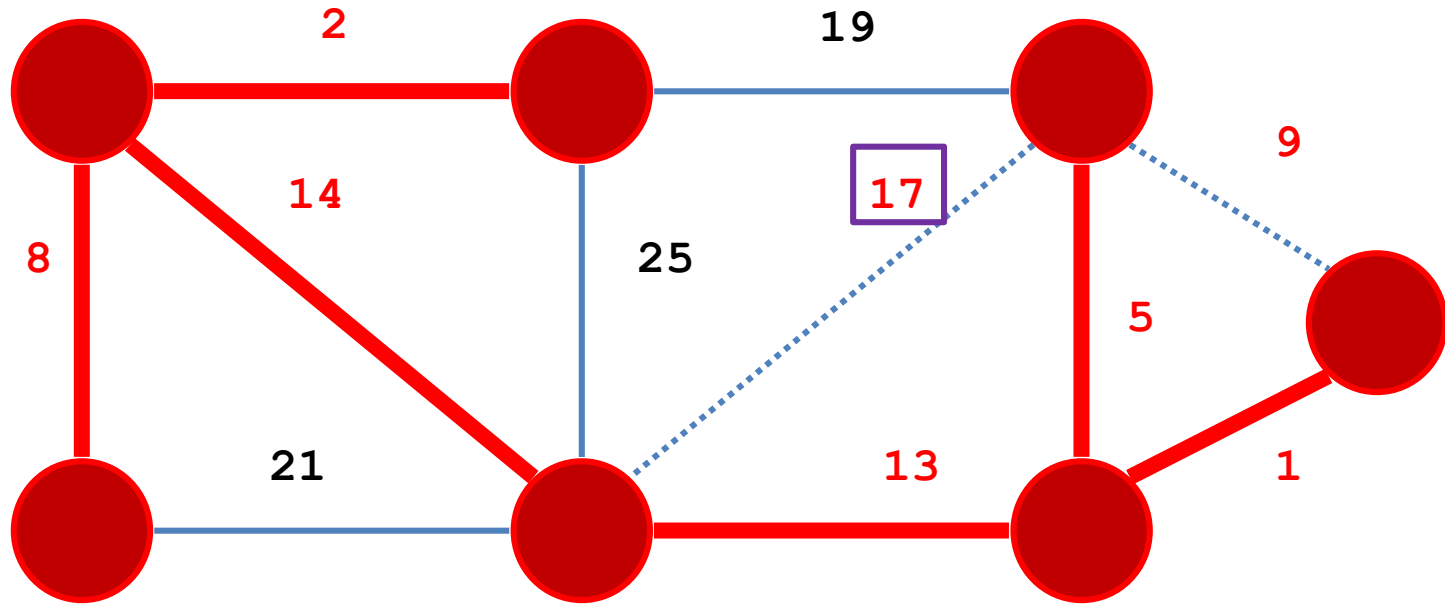
Exemplo:



$$\left. \begin{array}{l} w_{T_1} = 1 + 5 + 13 \\ w_{T_2} = 2 + 8 \end{array} \right\} w_{T_1 + T_2} = 19 + 10 + 14$$

1 – Algoritmo de Kruskal

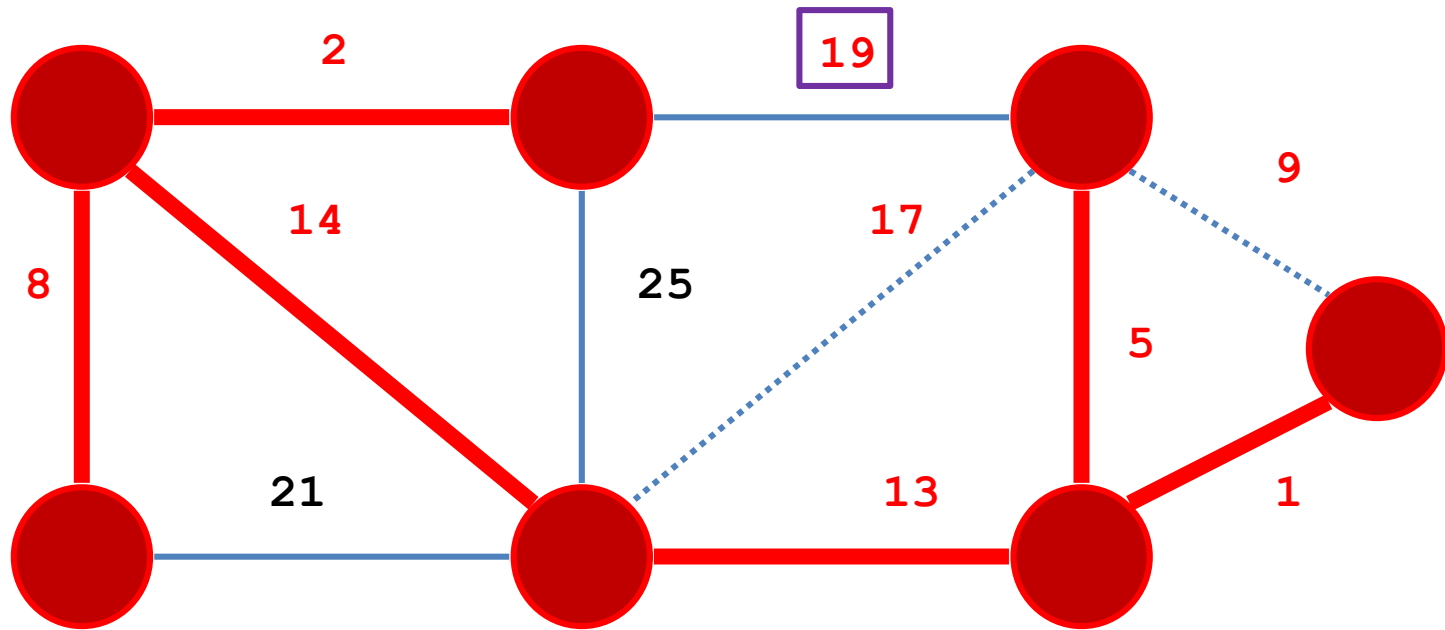
Exemplo:



$$\left. \begin{array}{l} w_{T_1} = 1 + 5 + 13 \\ w_{T_2} = 2 + 8 \end{array} \right\} w_{T_1 + T_2} = 19 + 10 + 14$$

1 – Algoritmo de Kruskal

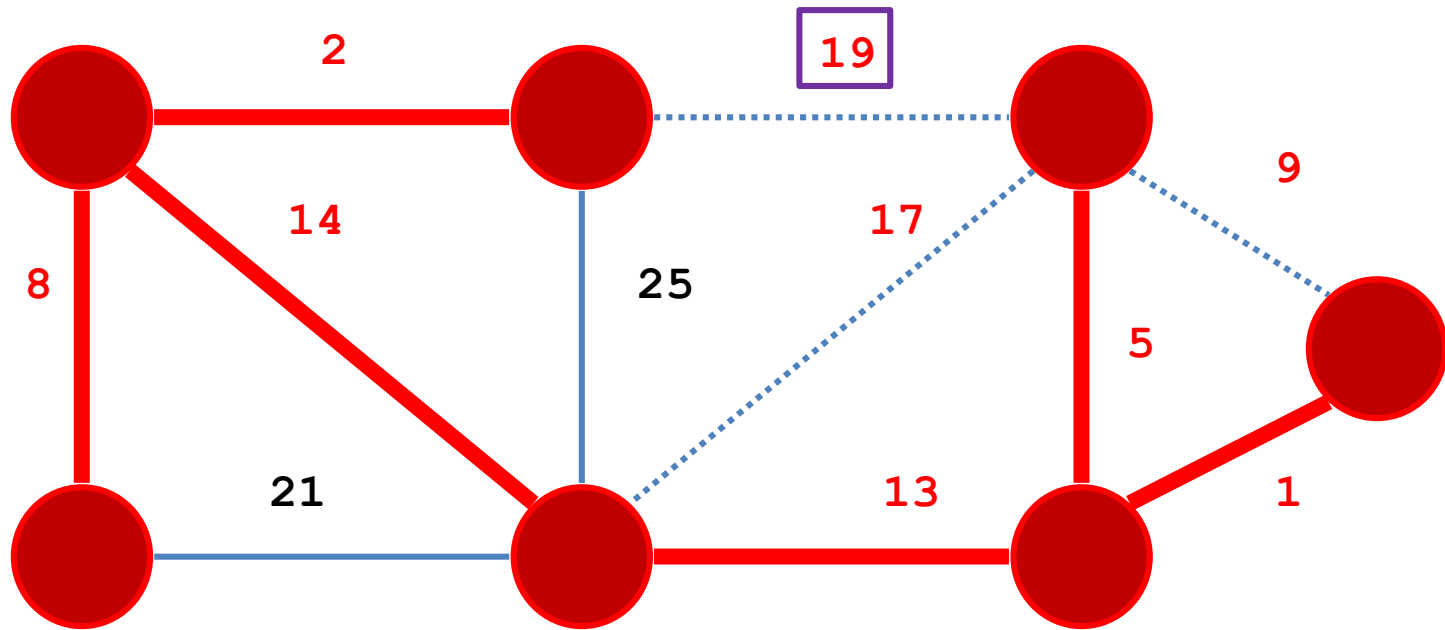
Exemplo:



$$\left. \begin{array}{l} w_{T_1} = 1 + 5 + 13 \\ w_{T_2} = 2 + 8 \end{array} \right\} w_{T_1 + T_2} = 19 + 10 + 14$$

1 – Algoritmo de Kruskal

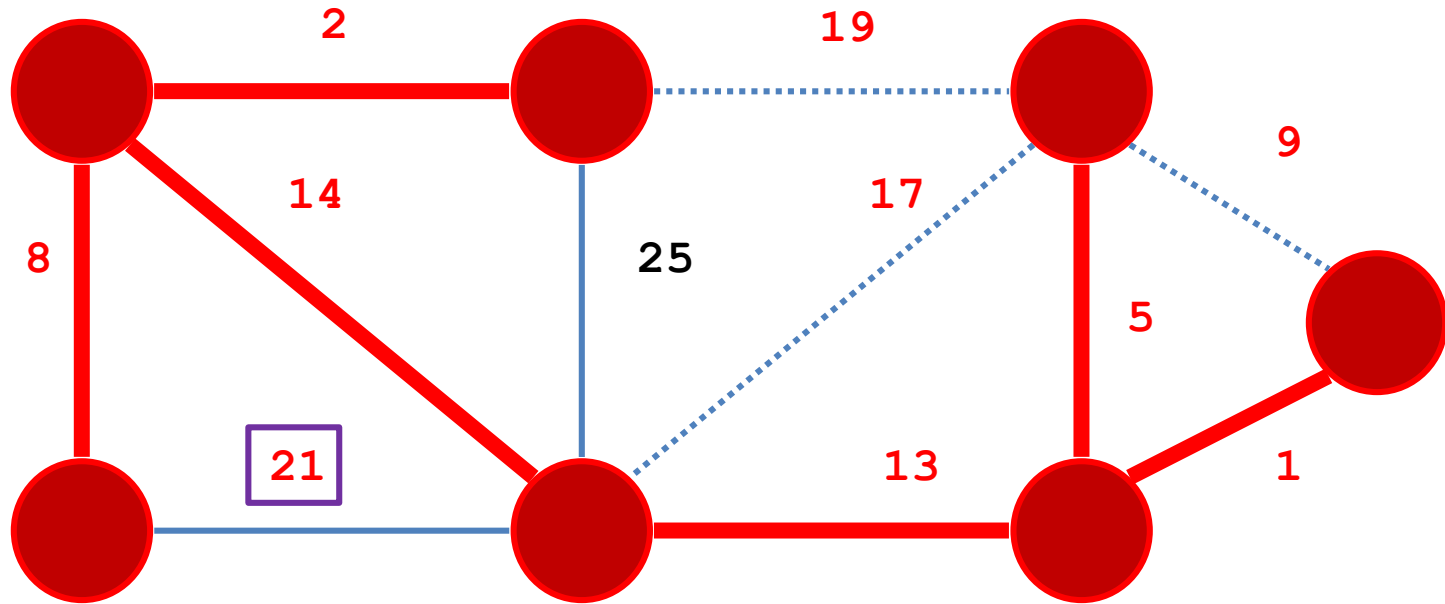
Exemplo:



$$\left. \begin{array}{l} w_{T_1} = 1 + 5 + 13 \\ w_{T_2} = 2 + 8 \end{array} \right\} w_{T_1 + T_2} = 19 + 10 + 14$$

1 – Algoritmo de Kruskal

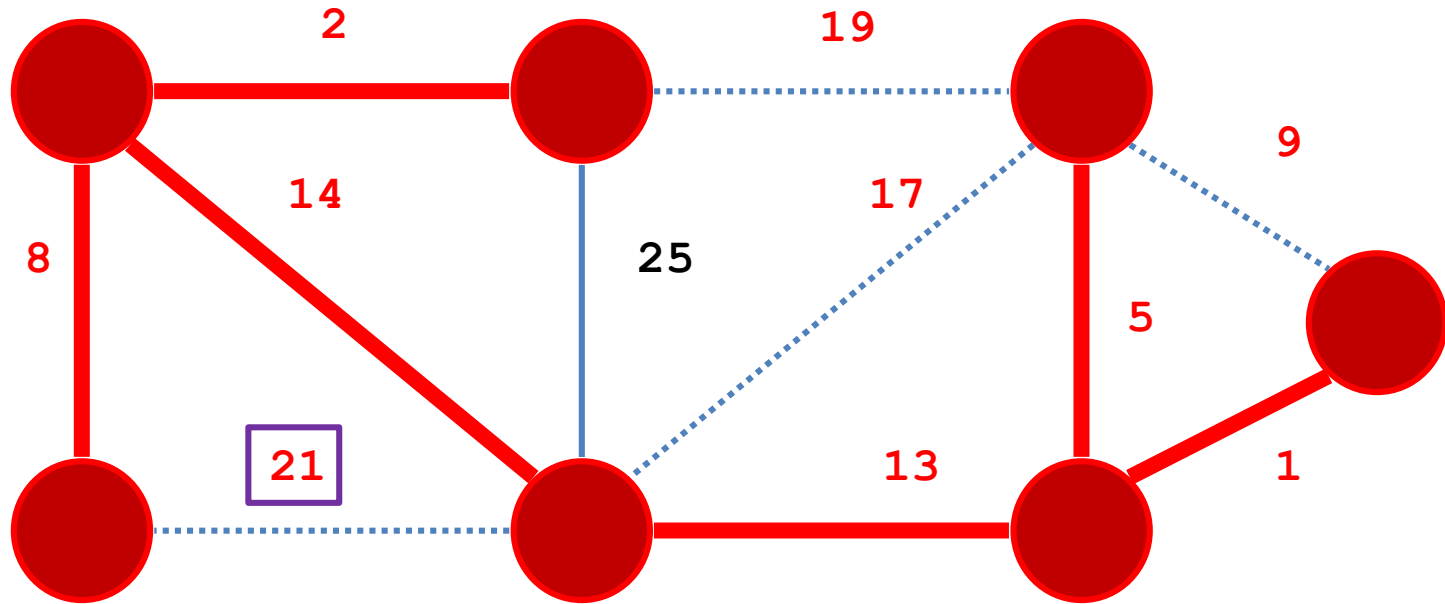
Exemplo:



$$\left. \begin{array}{l} w_{T_1} = 1 + 5 + 13 \\ w_{T_2} = 2 + 8 \end{array} \right\} w_{T_1 + T_2} = 19 + 10 + 14$$

1 – Algoritmo de Kruskal

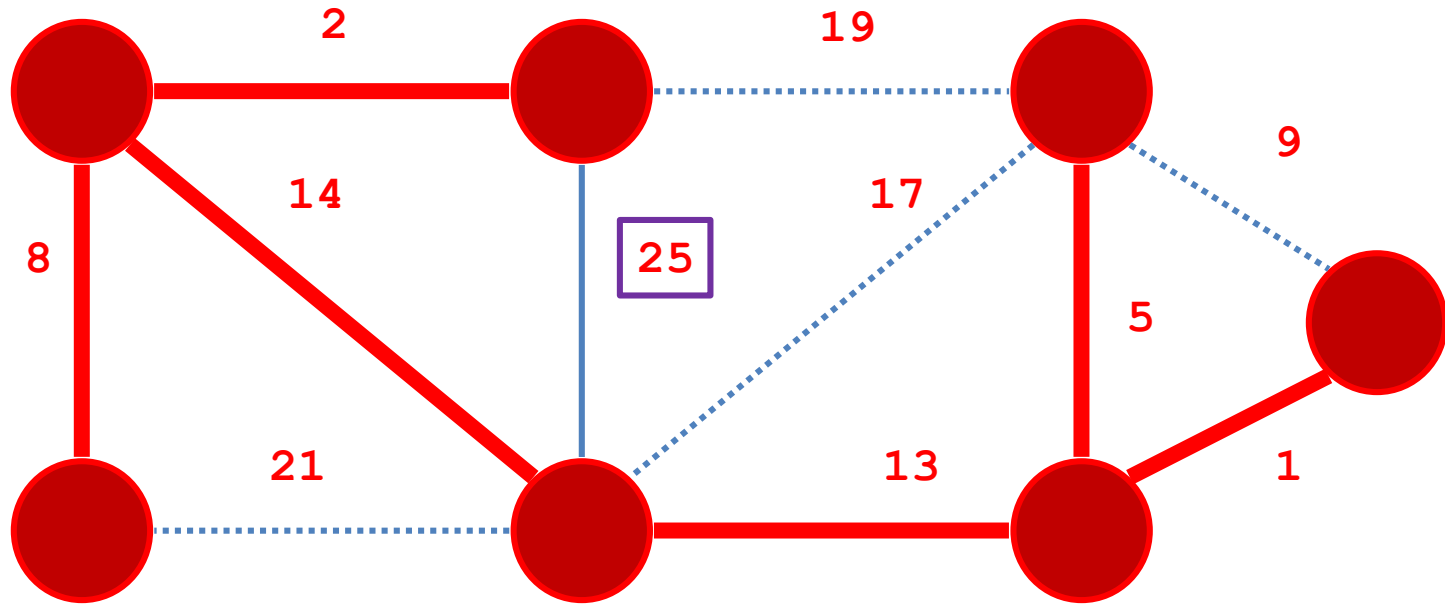
Exemplo:



$$\left. \begin{array}{l} w_{T_1} = 1 + 5 + 13 \\ w_{T_2} = 2 + 8 \end{array} \right\} w_{T_1 + T_2} = 19 + 10 + 14$$

1 – Algoritmo de Kruskal

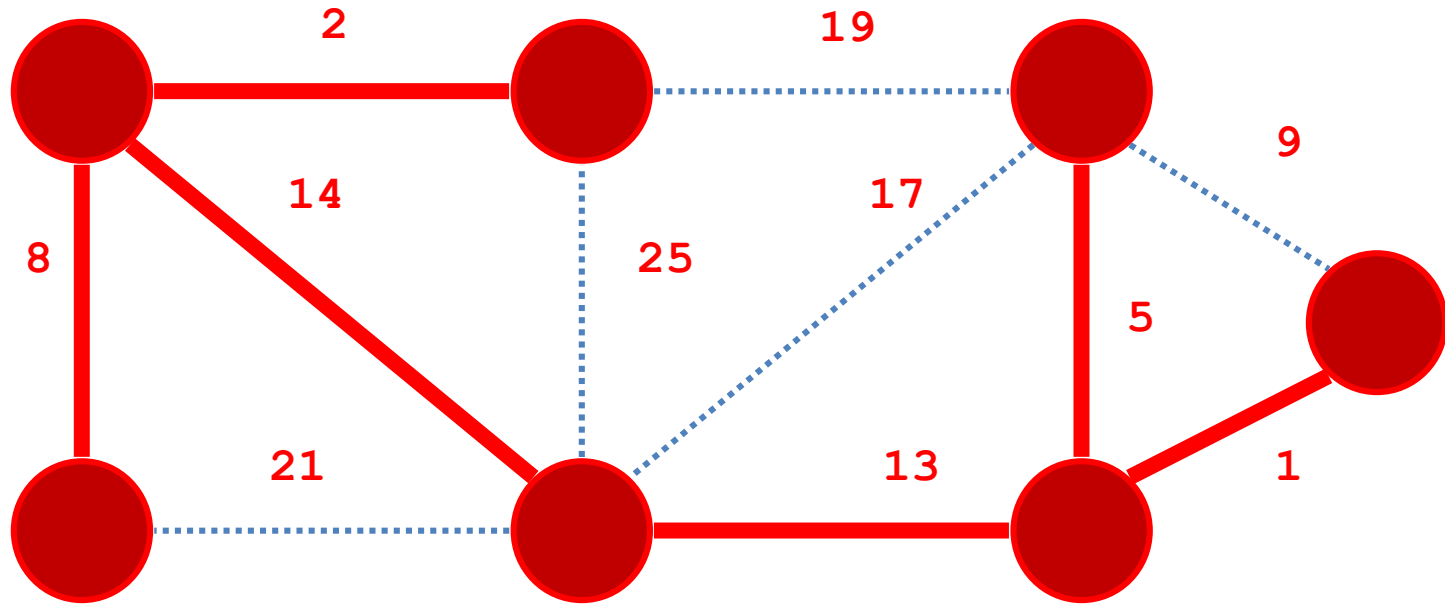
Exemplo:



$$\left. \begin{array}{l} w_{T_1} = 1 + 5 + 13 \\ w_{T_2} = 2 + 8 \end{array} \right\} w_{T_1 + T_2} = 19 + 10 + 14$$

1 – Algoritmo de Kruskal

Exemplo:



$$\left. \begin{array}{l} w_{T_1} = 1 + 5 + 13 \\ w_{T_2} = 2 + 8 \end{array} \right\} w_{T_1 + T_2} = 19 + 10 + 14 = 43$$

1 – Algoritmo de Kruskal

Algoritmo:

```
Kruskal() {  
1  A =  $\emptyset$ ;  
2  for cada vértice  $v \in V[G]$   
3      do MakeSet( $v$ );  
4  Ordene as arestas E de forma crescente ao peso w  
5  for cada aresta  $(u,v) \in E$  (ordenado)  
6      do if FindSet( $u$ )  $\neq$  FindSet( $v$ )  
7          then A = A  $\cup$  { $(u,v)$ };  
8  Union(FindSet( $u$ ), FindSet( $v$ ));  
9  return A;  
}
```

1 – Algoritmo de Kruskal

Algoritmo:

```
Kruskal() {
```

```
1  A =  $\emptyset$ ;
```

```
2  for cada vértice  $v \in V[G]$ 
```

```
3      do MakeSet( $v$ );
```

```
4  Ordene as arestas  $E$  de forma crescente ao peso  $w$ 
```

```
5  for cada aresta  $(u,v) \in E$  (ordenado)
```

```
6      do if FindSet( $u$ )  $\neq$  FindSet( $v$ )
```

```
7          then A = A  $\cup$  {  $(u,v)$  };
```

```
8  Union(FindSet( $u$ ), FindSet( $v$ ));
```

```
9  return A;
```

```
}
```

Inicializa o conjunto **A** como o conjunto vazio e cria **V** árvores, cada uma contendo 1 vértice.

1 – Algoritmo de Kruskal

Algoritmo:

```
Kruskal() {  
  1  A =  $\emptyset$ ;  
  2  for cada vértice  $v \in V[G]$   As arestas em E são ordenadas em  
  3      do MakeSet(v);          sequência crescente por peso.  
  4  Ordene as arestas E de forma crescente ao peso w  
  5  for cada aresta  $(u,v) \in E$  (ordenado)  
  6      do if FindSet(u)  $\neq$  FindSet(v)  
  7          then A = A  $\cup$  { (u,v) };  
  8  Union(FindSet(u), FindSet(v));  
  9  return A;  
}
```

1 – Algoritmo de Kruskal

Algoritmo:

```
Kruskal() {  
  1  A =  $\emptyset$ ;  
  2  for cada vértice  $v \in V[G]$   
  3    do MakeSet( $v$ ) ;  
  4  Ordene as arestas E de forma crescente ao peso w  
  5  for cada aresta  $(u, v) \in E$  (ordenado)  
  6    do if FindSet( $u$ )  $\neq$  FindSet( $v$ )  
  7      then A = A  $\cup$  {  $(u, v)$  } ;  
  8  Union(FindSet( $u$ ) , FindSet( $v$ ) ) ;  
  9  return A ;  
}
```

Verifica, para cada aresta (u, v) , se os pontos extremos u e v pertencem à mesma árvore. Se sim, a aresta é descartada pois **não é permitido ciclo**. Se não, a aresta é adicionada em **A** e os vértices são intercalados.

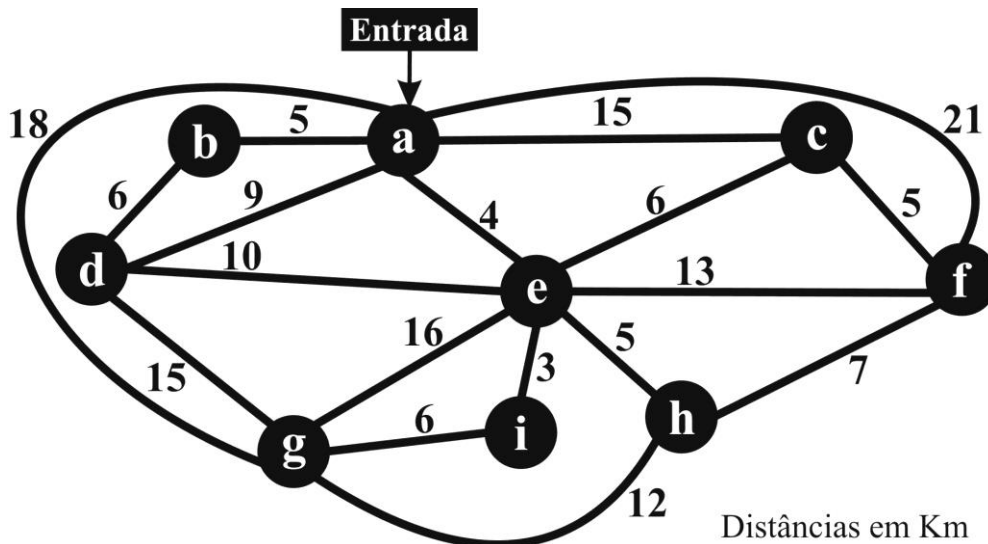
1 – Algoritmo de Kruskal

Exercício 1:

A figura abaixo mostra o esquema de uma velha rede de cabos de distribuição de energia elétrica que conecta 9 subestações de uma determinada cidade.

A **Cia. Elétrica** local precisa renovar a rede substituindo os cabos velhos por novos, e também enviar para reciclagem os cabos desnecessários.

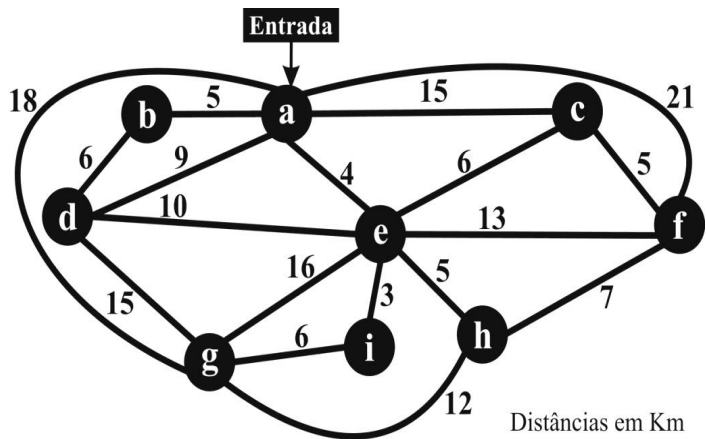
- a) Considerando a figura abaixo como $G = (V, E)$, monte a árvore geradora mínima e indique o seu peso mínimo, utilizando o algoritmo de *Kruskal*:



1 – Algoritmo de Kruskal

Exercício 1 (resolução):

- a) Considerando a figura abaixo como $G = (V, E)$, monte a árvore geradora mínima e indique o seu peso mínimo, utilizando o algoritmo de *Kruskal*:

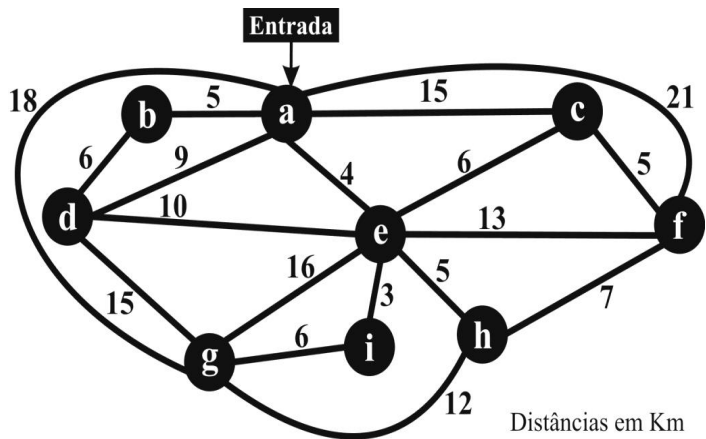


$$W_{t1}=3$$

1 – Algoritmo de Kruskal

Exercício 1 (resolução):

- a) Considerando a figura abaixo como $G = (V, E)$, monte a árvore geradora mínima e indique o seu peso mínimo, utilizando o algoritmo de *Kruskal*:

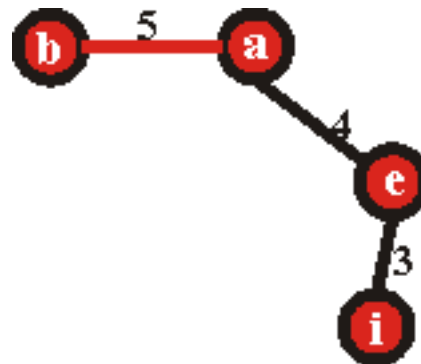
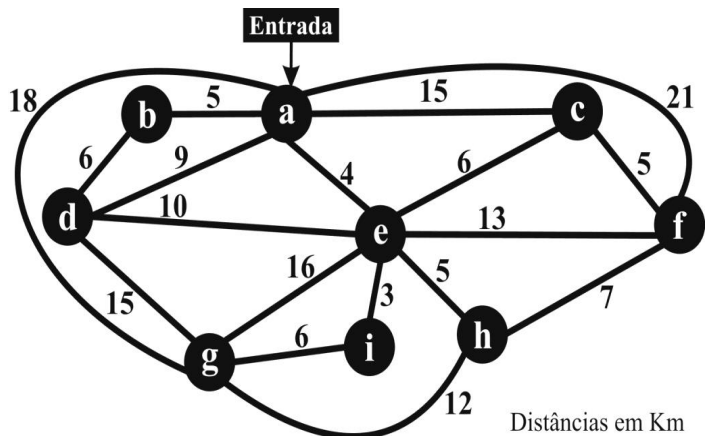


$$W_{t1} = 3 + 4$$

1 – Algoritmo de Kruskal

Exercício 1 (resolução):

- a) Considerando a figura abaixo como $G = (V, E)$, monte a árvore geradora mínima e indique o seu peso mínimo, utilizando o algoritmo de *Kruskal*:

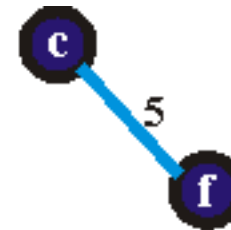
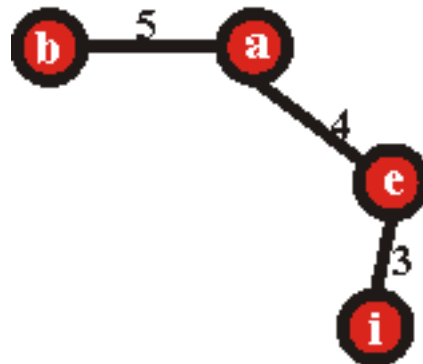
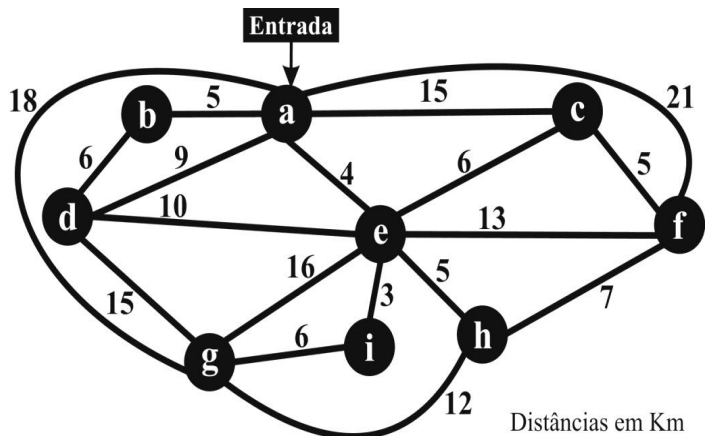


$$W_{t1} = 3 + 4 + 5$$

1 – Algoritmo de Kruskal

Exercício 1 (resolução):

- a) Considerando a figura abaixo como $G = (V, E)$, monte a árvore geradora mínima e indique o seu peso mínimo, utilizando o algoritmo de *Kruskal*:



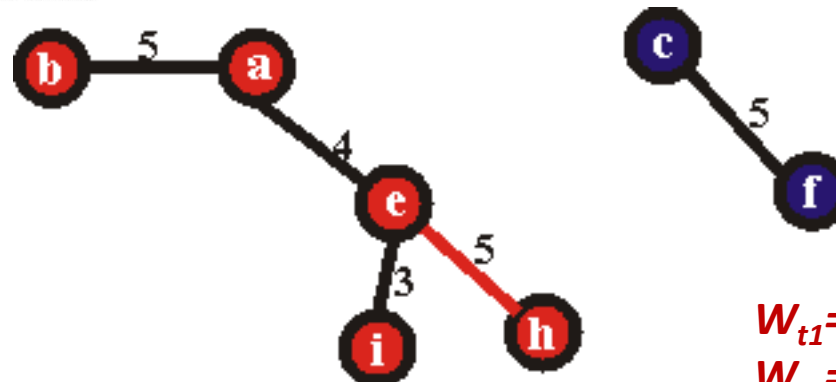
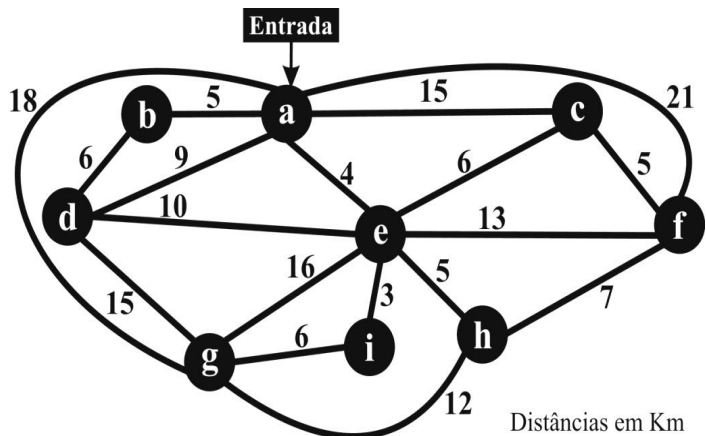
$$W_{t1} = 3 + 4 + 5$$

$$W_{t2} = 5$$

1 – Algoritmo de Kruskal

Exercício 1 (resolução):

- a) Considerando a figura abaixo como $G = (V, E)$, monte a árvore geradora mínima e indique o seu peso mínimo, utilizando o algoritmo de *Kruskal*:



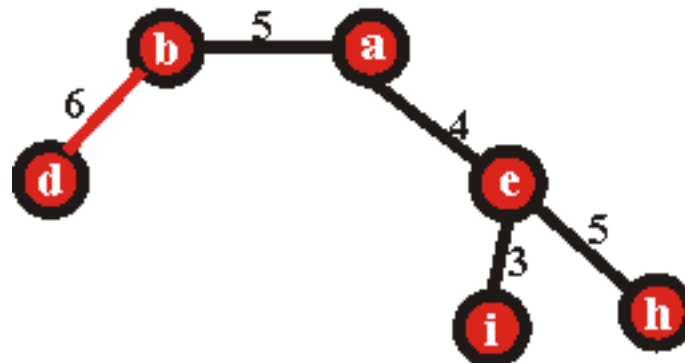
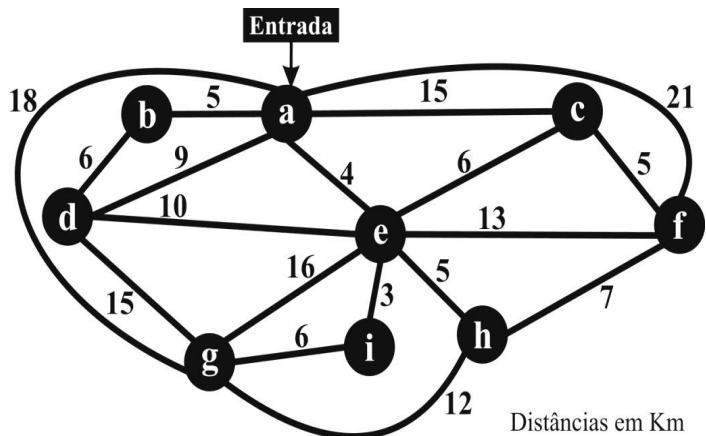
$$W_{t1} = 3 + 4 + 5 + 5$$

$$W_{t2} = 5$$

1 – Algoritmo de Kruskal

Exercício 1 (resolução):

- a) Considerando a figura abaixo como $G = (V, E)$, monte a árvore geradora mínima e indique o seu peso mínimo, utilizando o algoritmo de *Kruskal*:



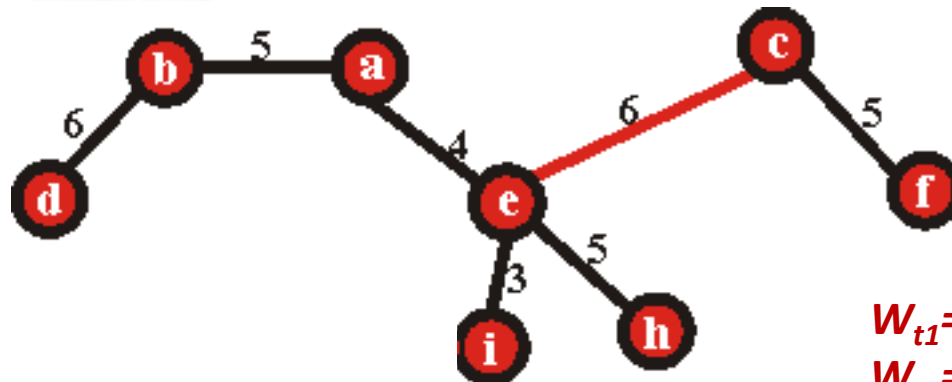
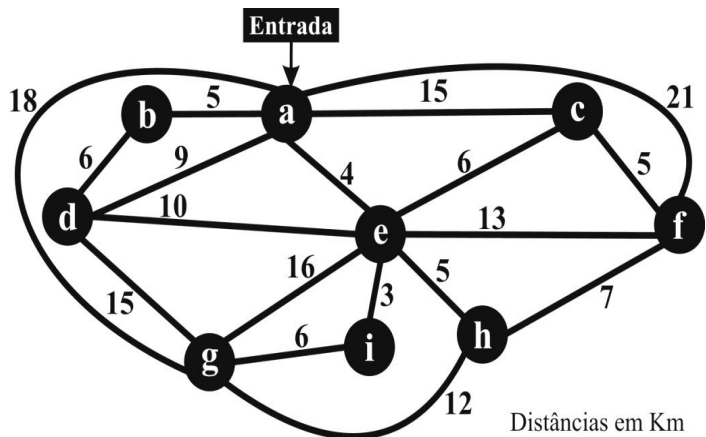
$$W_{t1} = 3 + 4 + 5 + 5 + 6$$

$$W_{t2} = 5$$

1 – Algoritmo de Kruskal

Exercício 1 (resolução):

- a) Considerando a figura abaixo como $G = (V, E)$, monte a árvore geradora mínima e indique o seu peso mínimo, utilizando o algoritmo de *Kruskal*:



$$W_{t1} = 3 + 4 + 5 + 5 + 6$$

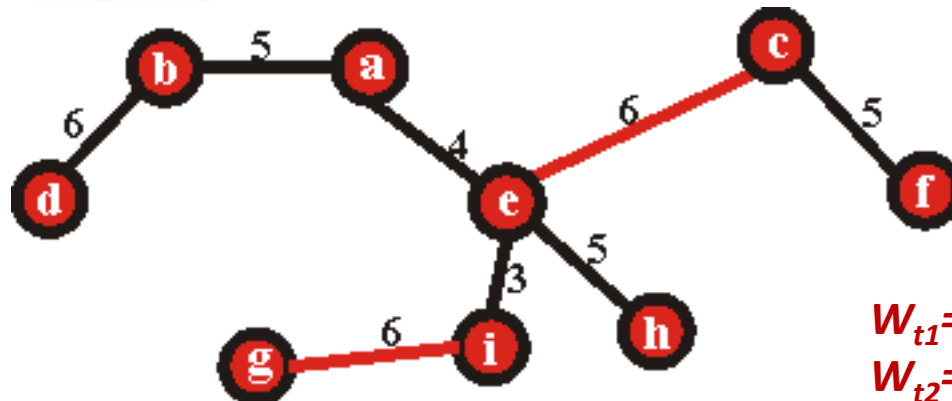
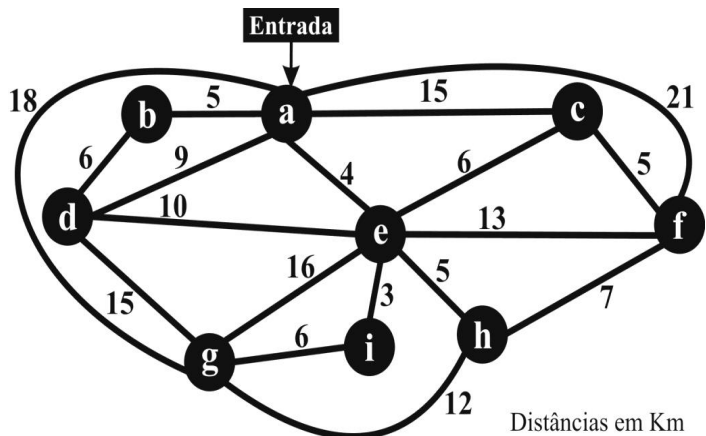
$$W_{t2} = 5$$

$$W_{t1+t2} = 23 + 5 + 6$$

1 – Algoritmo de Kruskal

Exercício 1 (resolução):

- a) Considerando a figura abaixo como $G = (V, E)$, monte a árvore geradora mínima e indique o seu peso mínimo, utilizando o algoritmo de *Kruskal*:



$$W_{t_1} = 3 + 4 + 5 + 5 + 6$$

$$W_{t_2} = 5$$

$$W_{t_1+t_2} = 23 + 5 + 6 + 6 = 40$$

Peso mínimo é 40

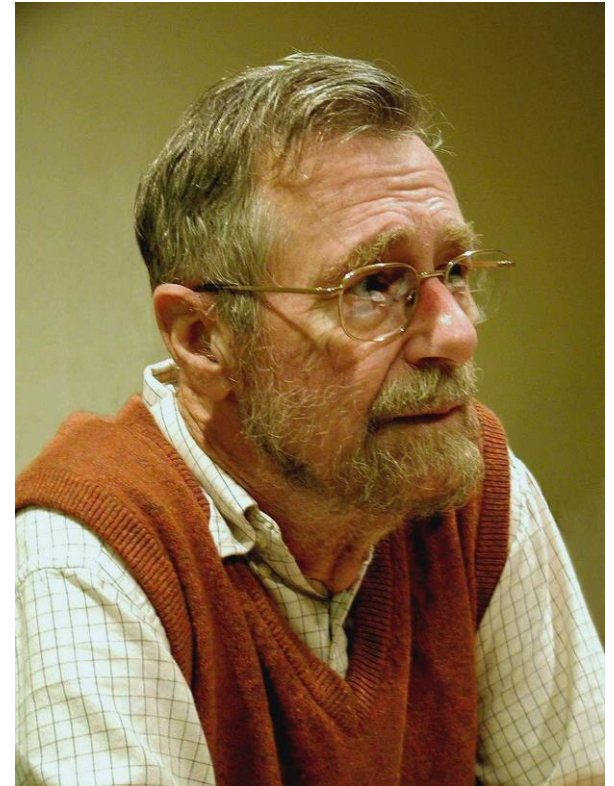
2

Algoritmo de Dijkstra

2 – Algoritmo de Dijkstra

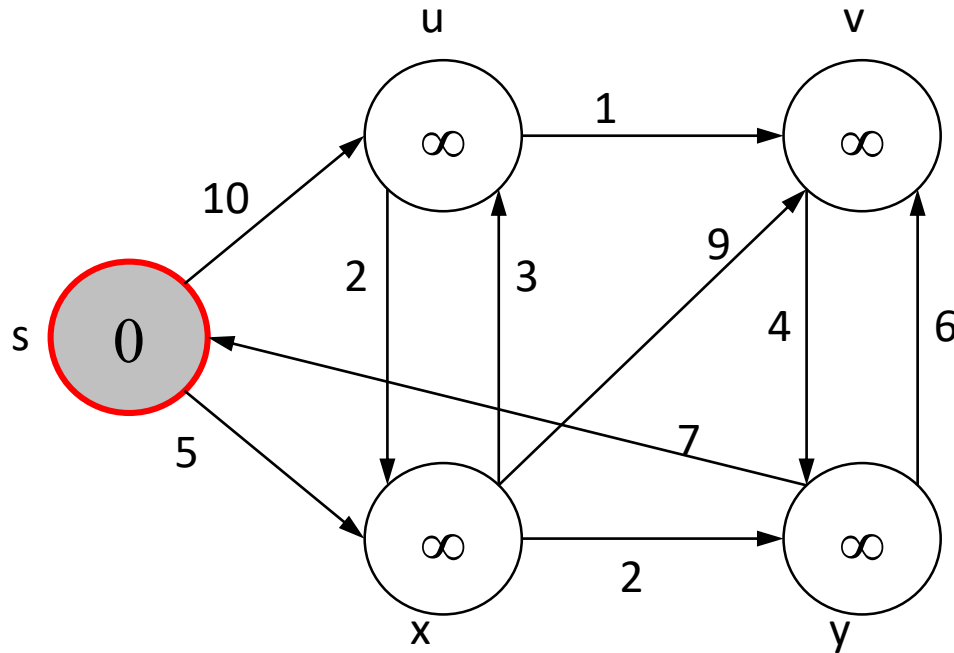
Características:

- O **algoritmo de Dijkstra**, concebido pelo cientista da computação holandês Edsger Dijkstra em 1956 e publicado em 1959.
- É um algoritmo de busca, guloso, que resolve o problema de caminhos mais curtos de única origem em um grafo orientado ponderado.
- As arestas não podem ter pesos negativos.



2 – Algoritmo de Dijkstra

Exemplo:

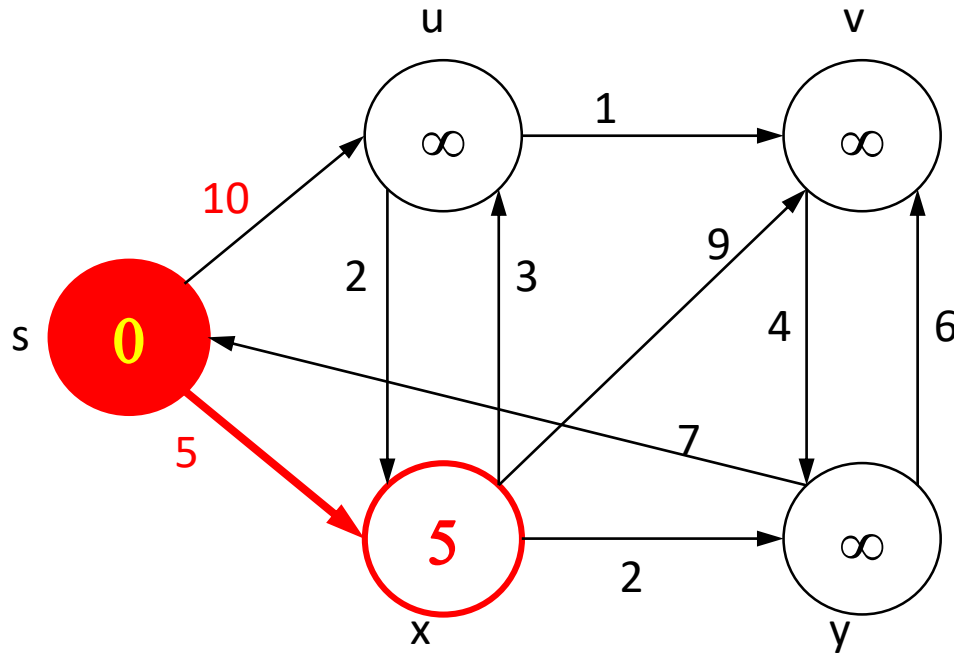


Inicialmente todos os vértices têm um custo infinito, exceto s (a raiz da busca) que tem valor 0:

Vértices	s	u	v	x	y
Estimativas	0	∞	∞	∞	∞
Precedentes	-	-	-	-	-
Fechado	Não	-	-	-	-

2 – Algoritmo de Dijkstra

Exemplo



Selecione o vértice aberto de estimativa mínima: **s**

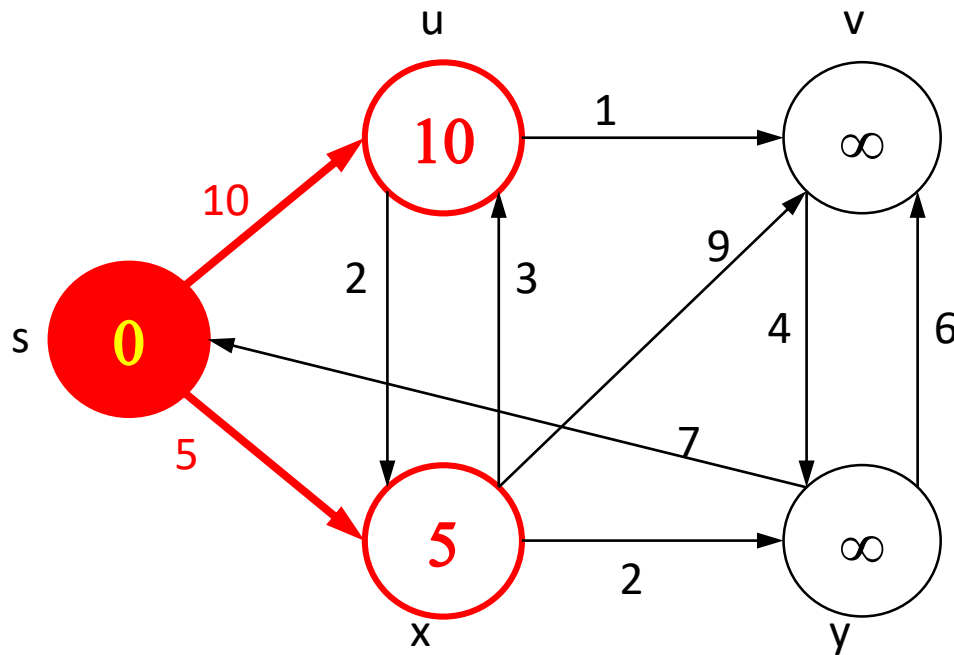
Feche **s**.

Recalcule as estimativas de **u** e **x**.

Vértices	s	u	v	x	y
Estimativas	0	∞	∞	5	∞
Precedentes	-	-	-	s	-
Fechado	Sim	-	-	Não	-

2 – Algoritmo de Dijkstra

Exemplo:



Selecione o vértice aberto de estimativa mínima: **s**

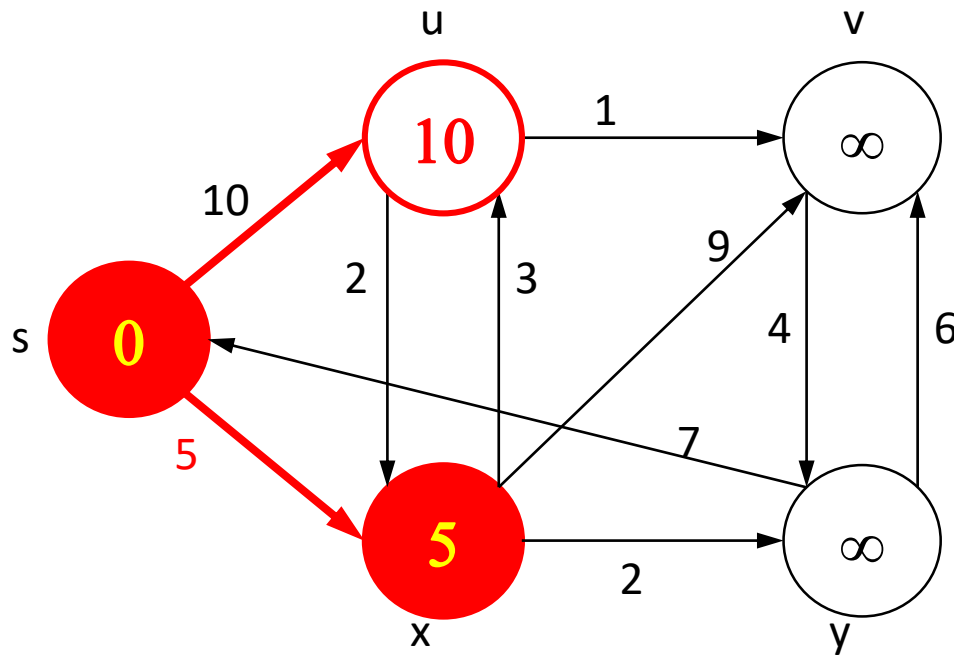
Feche **s**.

Recalcule as estimativas de **u** e **x**.

Vértices	s	u	v	x	y
Estimativas	0	10	∞	5	∞
Precedentes	-	s	-	s	-
Fechado	Sim	Não	-	Não	-

2 – Algoritmo de Dijkstra

Exemplo:



Selecione o vértice aberto de estimativa mínima: **x**

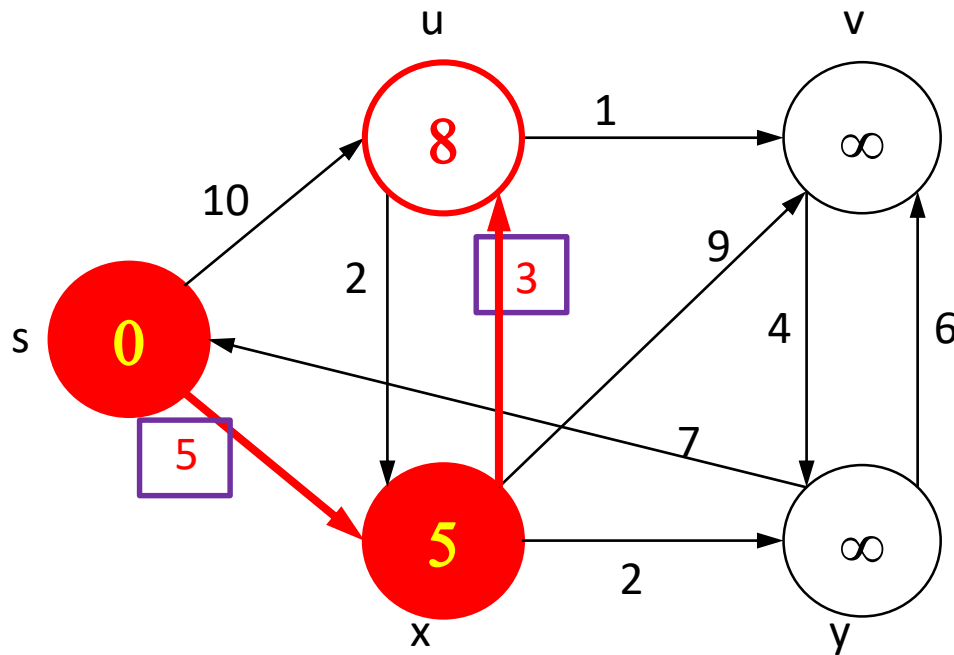
Feche **x**.

Recalcule as estimativas de **u**, **v** e **y**.

Vértices	s	u	v	x	y
Estimativas	0	10	∞	5	∞
Precedentes	-	s	-	s	-
Fechado	Sim	Não	-	Sim	-

2 – Algoritmo de Dijkstra

Exemplo:



Selecione o vértice aberto de estimativa mínima: **x**

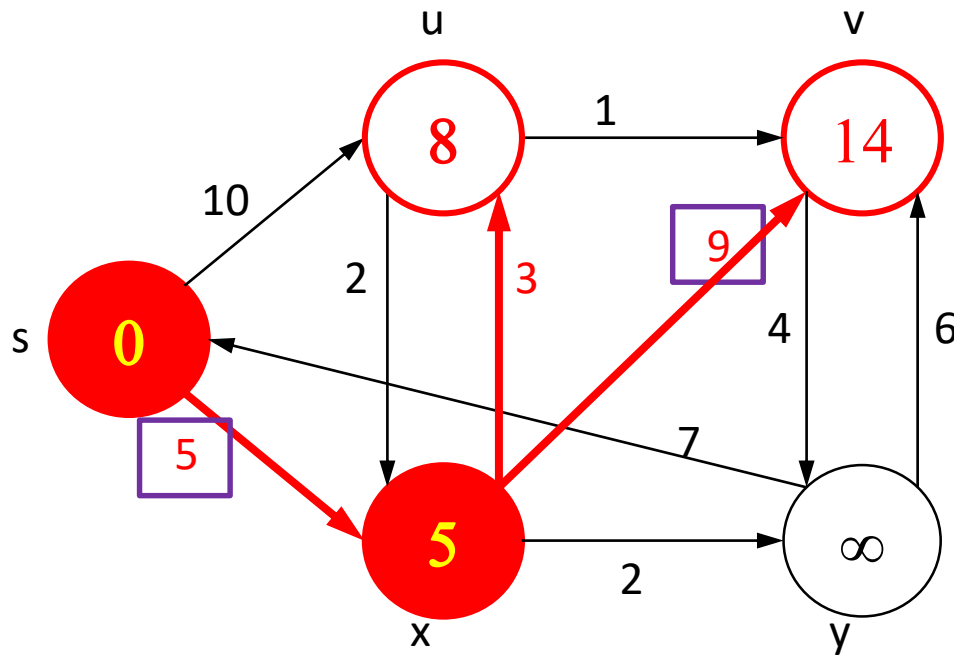
Feche **x**.

Recalcule as estimativas de **u**, **v** e **y**.

Vértices	s	u	v	x	y
Estimativas	0	10 8	∞	5	∞
Precedentes	-	x	-	s	-
Fechado	Sim	Não	-	Sim	-

2 – Algoritmo de Dijkstra

Exemplo:



Selecione o vértice aberto de estimativa mínima: **x**

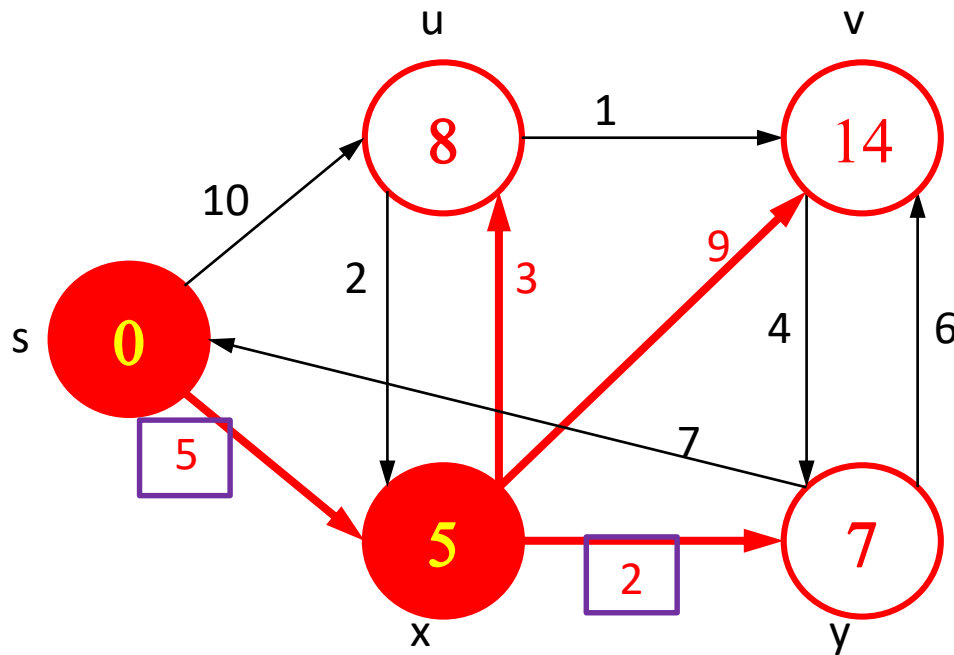
Feche **x**.

Recalcule as estimativas de **u**, **v** e **y**.

Vértices	s	u	v	x	y
Estimativas	0	10 8	14	5	∞
Precedentes	-	x	x	s	-
Fechado	Sim	Não	Não	Sim	-

2 – Algoritmo de Dijkstra

Exemplo:



Selecione o vértice aberto de estimativa mínima: **x**

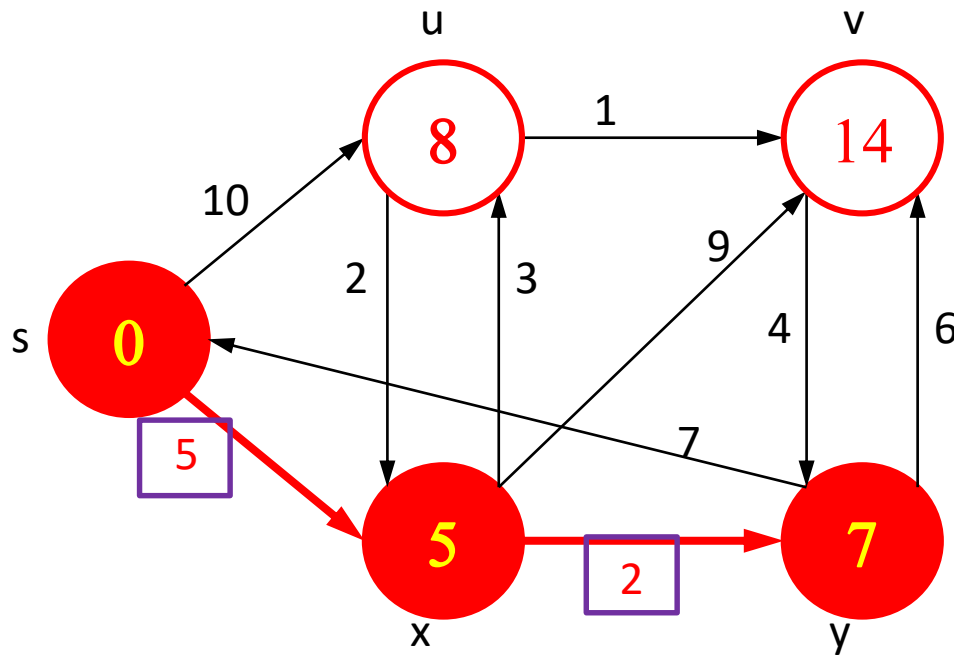
Feche **x**.

Recalcule as estimativas de **u**, **v** e **y**.

Vértices	s	u	v	x	y
Estimativas	0	10 8	14	5	7
Precedentes	-	x	x	s	x
Fechado	Sim	Não	Não	Sim	Não

2 – Algoritmo de Dijkstra

Exemplo:



Selecione o vértice aberto de estimativa mínima: **y**

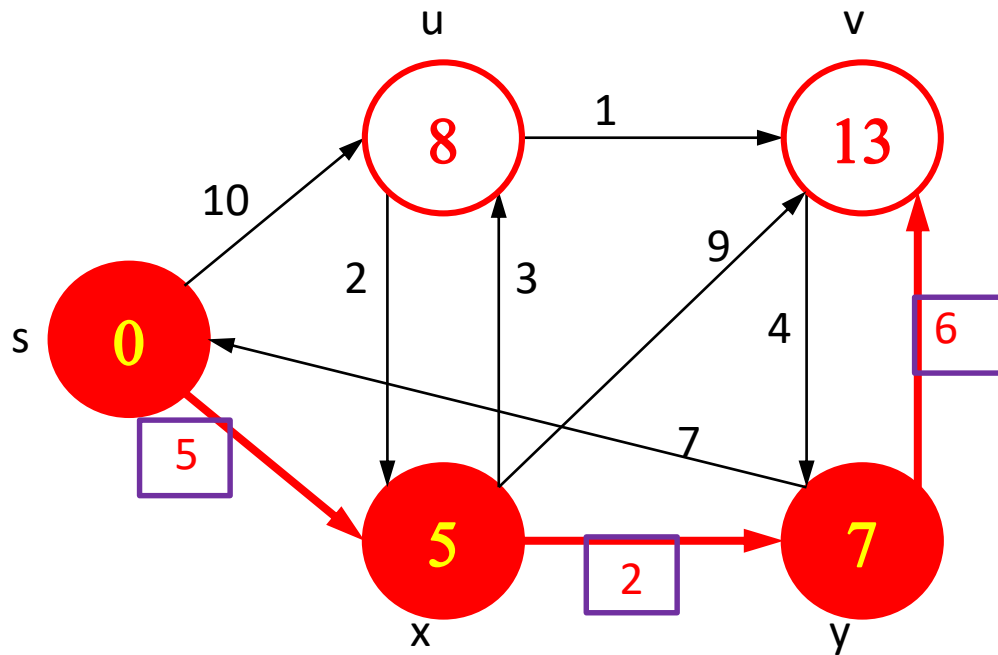
Feche **y**.

Recalcule as estimativas de **v**.

Vértices	s	u	v	x	y
Estimativas	0	10 8	14	5	7
Precedentes	-	x	x	s	x
Fechado	Sim	Não	Não	Sim	Sim

2 – Algoritmo de Dijkstra

Exemplo:



Selecione o vértice aberto de estimativa mínima: **y**

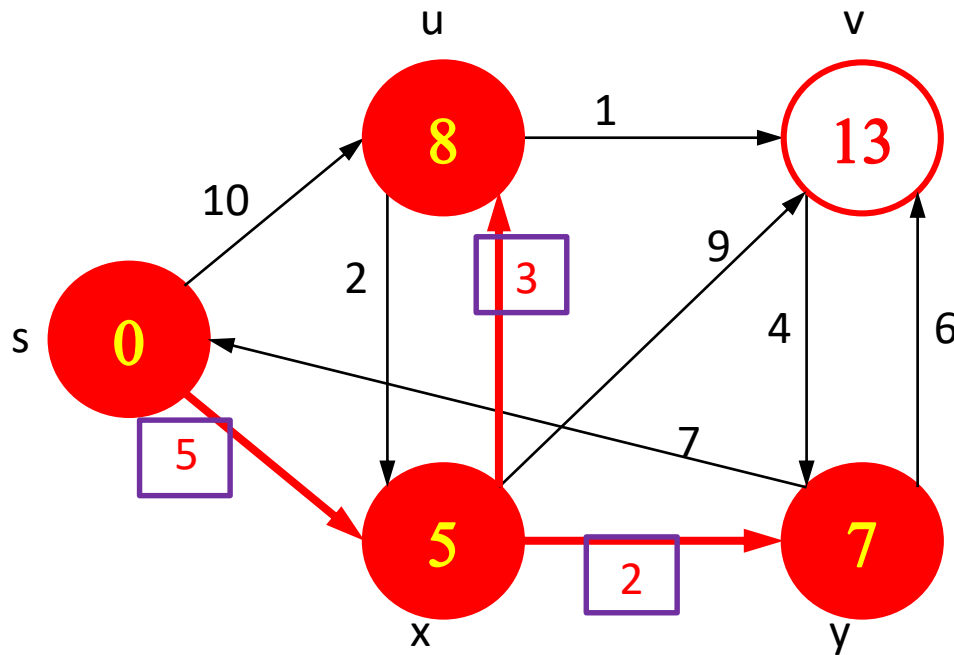
Feche **y**.

Recalcule as estimativas de **v**.

Vértices	s	u	v	x	y
Estimativas	0	10 8	14 13	5	7
Precedentes	-	x	y	s	x
Fechado	Sim	Não	Não	Sim	Sim

2 – Algoritmo de Dijkstra

Exemplo:



Selecione o vértice aberto de estimativa mínima: **u**

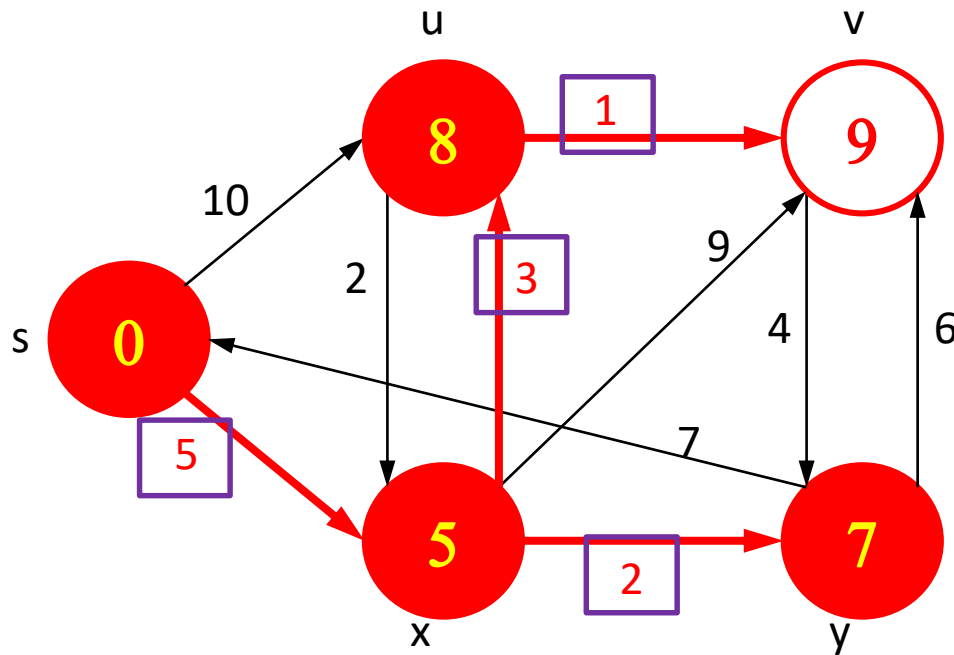
Feche **u**.

Recalcule as estimativas de **v**.

Vértices	s	u	v	x	y
Estimativas	0	8	14 13	5	7
Precedentes	-	x	y	s	x
Fechado	Sim	Sim	Não	Sim	Sim

2 – Algoritmo de Dijkstra

Exemplo:



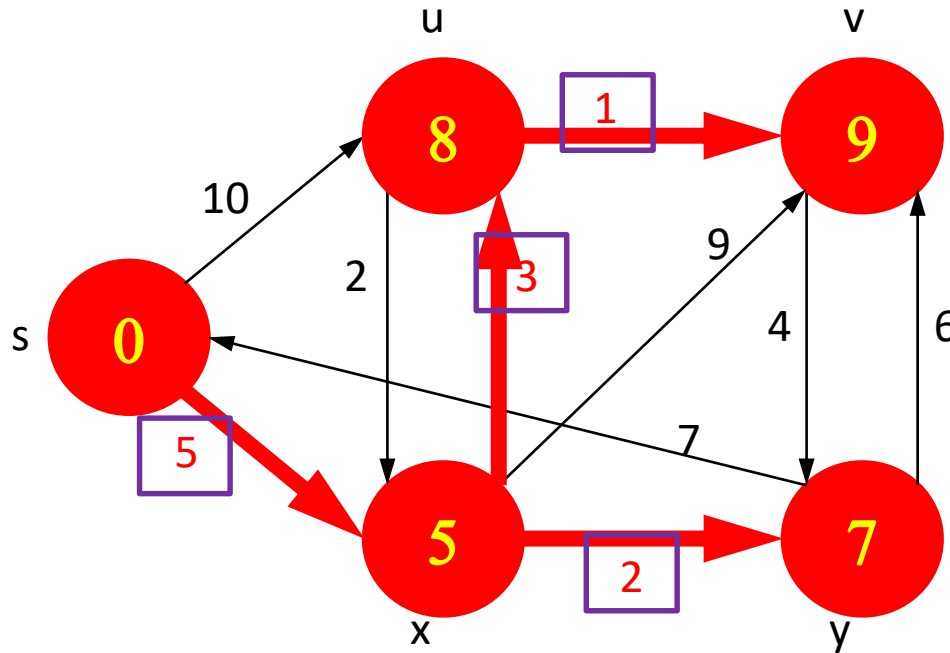
Selecione o vértice aberto de estimativa mínima: **v**

Feche **v**.

Vértices	s	u	v	x	y
Estimativas	0	8	13 9	5	7
Precedentes	-	x	u	s	x
Fechado	Sim	Sim	Não	Sim	Sim

2 – Algoritmo de Dijkstra

Exemplo:



Selecione o vértice aberto de estimativa mínima: **v**

Feche **v**.

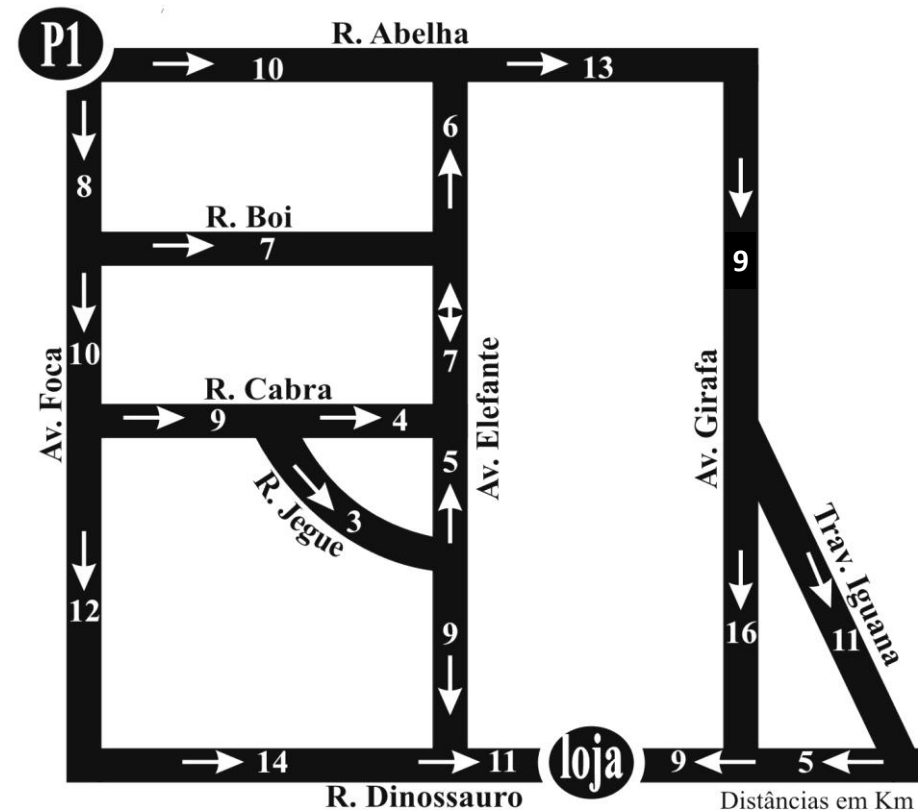
Vértices	s	u	v	x	y
Estimativas	0	8	9	5	7
Precedentes	-	x	u	s	x
Fechado	Sim	Sim	Sim	Sim	Sim

2 – Algoritmo de Dijkstra

Exercício 2:

A figura abaixo mostra o mapa de ruas de uma determinada cidade. Uma loja situada na **R. Dinossauro** está sendo assaltada, e enviou um pedido de socorro à polícia. Uma viatura policial estacionada no ponto **P1** será destacada para atender ao urgente chamado:

- Monte um grafo $G = (V, E)$ que represente o cenário do enunciado, considerando cada esquina como um vértice (e dê rótulos com letras do alfabeto a eles) e as ruas, travessas e avenidas como arestas ponderadas.
- Monte a árvore mínima utilizando o algoritmo de **Dijkstra** e mostre a menor distância que a viatura policial percorrerá de P1 até a Loja.

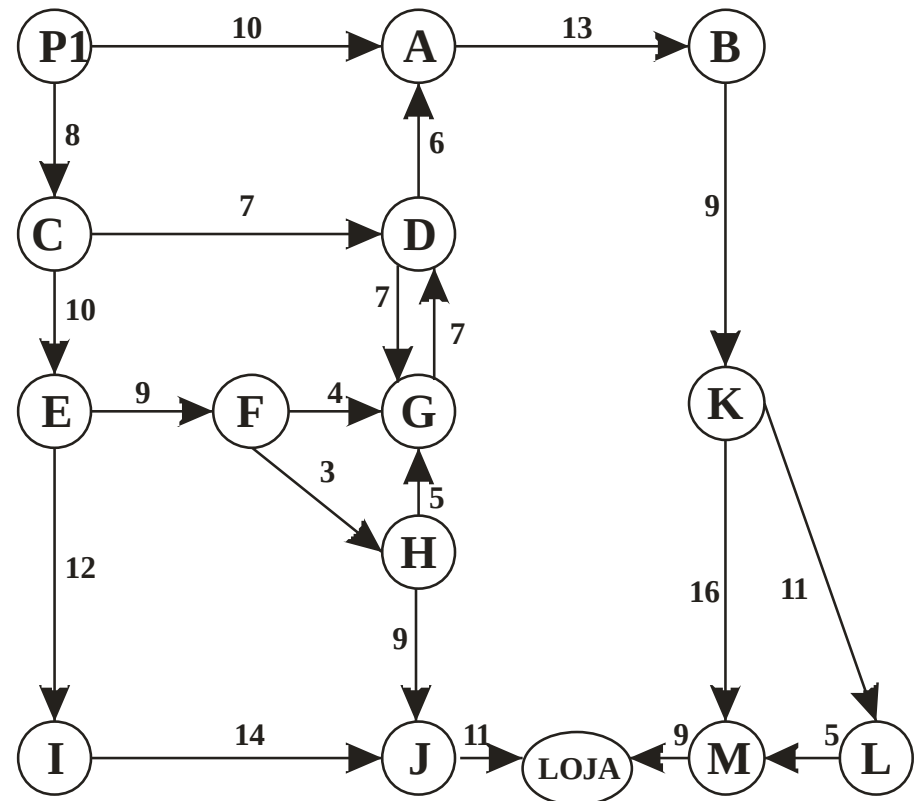


2 – Algoritmo de Dijkstra

Exercício 2 (resolução):

A figura abaixo mostra o mapa de ruas de uma determinada cidade. Uma loja situada na **R. Dinossauro** está sendo assaltada, e enviou um pedido de socorro à polícia. Uma viatura policial estacionada no ponto **P1** será destacada para atender ao urgente chamado:

- a) Monte um grafo $G = (V, E)$ que represente o cenário do enunciado, considerando cada esquina como um vértice (e dê rótulos com letras do alfabeto a eles) e as ruas, travessas e avenidas como arestas ponderadas.
- b) Monte a árvore mínima utilizando o algoritmo de **Dijkstra** e mostre a menor distância que a viatura policial percorrerá de P1 até a Loja.

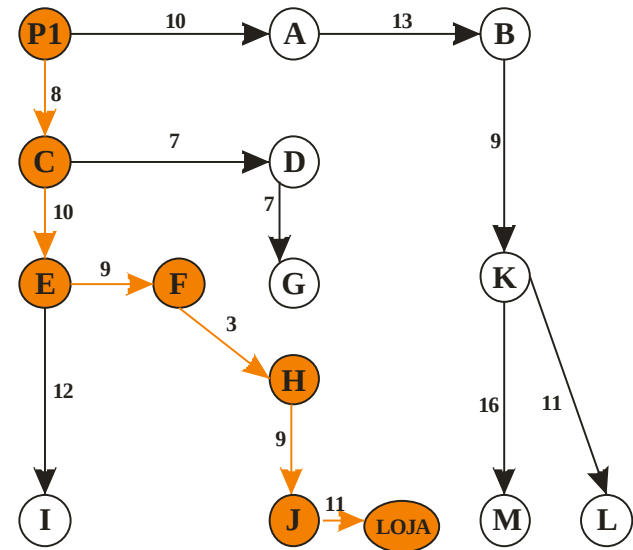


2 – Algoritmo de Dijkstra

Exercício 2 (resolução):

A figura abaixo mostra o mapa de ruas de uma determinada cidade. Uma loja situada na **R. Dinossauro** está sendo assaltada, e enviou um pedido de socorro à polícia. Uma viatura policial estacionada no ponto **P1** será destacada para atender ao urgente chamado:

- a) Monte um grafo $G = (V, E)$ que represente o cenário do enunciado, considerando cada esquina como um vértice (e dê rótulos com letras do alfabeto a eles) e as ruas, travessas e avenidas como arestas ponderadas.
- b) Monte a árvore mínima utilizando o algoritmo de **Dijkstra** e mostre a menor distância que a viatura policial percorrerá de P1 até a Loja.



Vértices	P1	A	C	D	E	B	G	F	I	K	H	J	L	M	LOJA	
Estimativas	0	10	8	15	18	23	22	27	30	32	30	44	39	43	48	50
Precedentes	-	P1	P1	C	C	A	D	E	E	B	F	↑ H	K	K	J	
Fechado	S	S	S	S	S	S	S	S	S	S	S	S	S	S	S	

Bibliografias

- i. CORMEN, LEISERSON, RIVEST, STEIN, **Algoritmos: Teoria e Prática**. 2ª ed. Rio de Janeiro: Campus, 2002: Capítulo 23.
- ii. TONIDANDEL, Flávio, Slides das aulas de Algoritmos Computacionais. FEI, 2011.
- iii. <http://www.inf.ufsc.br/grafos/temas/custo-minimo/dijkstra.html>
- iv. http://www.ime.usp.br/~pf/algoritmos_para_grafos/aulas/dijkstra.html
- v. <http://www.lcad.icmc.usp.br/~nonato/ED/Dijkstra/node84.html>
- vi. http://en.wikipedia.org/wiki/Prim%27s_algorithm
- vii. <http://www.ic.unicamp.br/~meidanis/courses/mo417/2003s1/aulas/2003-05-16.html>
- viii. <https://sites.google.com/site/tecprojalgoritmos/problemas/prim-e-kruskal>